Contents

Introduction

1.1 N-body approach for modelling structure formation: the main idea

The Universe is thought to be composed of dark matter and normal matter (the baryons), photons, neutrinos, etc. For the widest range of plausible dark matter (DM) candidates (from axions of mass $\sim 10^{-6} - 10^{-3}$ eV to $\sim 10^{21} - 10^{25}$ eV wimpzillas; see, e.g., Kolb et al. (1999); Roszkowski (1999)), their expected number density is[1] $n_{\rm dm} \sim (10^{52} - 10^{83})(1+\delta)\Omega_{\rm dm} h^2$ Mpc^{-3}, where δ is overdensity of matter. Thus, the number of actual dark matter particles is much beyond what is feasible to model numerically (currently this limit is $\sim 10^{12}$ particles). For a mix of particles of different types, evolution of the phase space density of species i is governed by the Boltzmann equation (the mass conservation or continuity equation in phase space):

$$\frac{\partial f_i}{\partial t} + \dot{\mathbf{x}}\frac{\partial f_i}{\partial \mathbf{x}} + \ddot{\mathbf{x}}\frac{\partial f_i}{\partial \dot{\mathbf{x}}} = C\left[f_i, f_j\right];$$

where $\mathbf{x}$ are positions and $\dot{\mathbf{x}}$ are velocities of a given volume element of phase space, and $C[f_i, f_j]$ is the collision integral, describing possible particle collisions, which can scatter particles in and out of a phase-space volume. The form of C depends on the specific properties of particles in the system (i.e., how they scatter, whether they can be created or annihilated, etc.). For collisionless dark matter the integral is set to zero, $C = 0$, which corresponds to the collisionless Boltzmann equation (CBE).

Dark matter candidates in structure formation scenarios typically are thought to have self-interaction cross sections of order $\sim 3\times10^{-26}\,{\rm cm}^{-3}\,s^{-1}/v \sim 10^{-36}\,{\rm cm}^2$ (the fact that results in so-called WIMP miracle), where $v \sim c$ is the typical speed of particle motion at freeze-out in the early universe. For a typical mass range considered for WIMP-like dark matter, $m_X c^2 \sim 10 - 100\,{\rm GeV}$, the corresponding mean free path is $l = (n\sigma)^{-1} \sim 10^{18} - 10^{19}\,h^{-2}(0.3/\Omega)(1+\delta)^{-1}\,{\rm Mpc}$. The mean free path for interaction of dark matter with baryon particles is at least several orders of magnitude larger than the mean free path for self-interaction. This large mean free path is justification for the standard assumption that dark matter in cosmological simulations can be treated as a collisionless system.[2] Similar arguments can be applied for stars in galaxies (e.g., Binney & Tremaine, 2008), so that stellar particles in galaxy formation simulations are also treated as collisionless system.

The Boltzmann equation is six-dimensional (phase space of thee coordinates and three velocities). If distribution of particles is assumed to be uniform in space or if deviations from uniformity are small, so that linear approximation is valid, the Boltzmann equation can be solved directly. This is indeed done in codes describing evolution of the primordial matter and linear perturbations to solve for the initial power spectrum and properties of the CMB temperature anisotropies. For the general case of nonlinear structure formation direct solution of the Boltzmann equation with sufficient resolution is not feasible (for a recent attempts to solve the Boltzmann equation directly with a limited resolution in the context of gravitational dynamics, see). Therefore, to model the evolution of dark matter distribution with a much smaller limited number of particles N-body approach is used.

In the N-body method the initial phase space is split into N small volume elements with associated mass treated as particles. The full solution of the CBE, as a solution of any ODE, can be represented by

[1] Here and throughout, we parameterize the Hubble constant at the present epoch as $H_0 = 100h$ km s^{-1} Mpc^{-1}. The present-day density of different components in the Universe is measured in terms of the critical density, $\rho_c = 3H_0^2/8\pi G = 1.8788\times10^{-29}h^2$ g cm^{-3} $= 2.7751\times10^{11}h^2$ M$_\odot$ Mpc^{-3}, and will be denoted as $\Omega_i = \rho_i/\rho_c$, where i is the component-specific subscript.

[2] Note that dark matter may have self-interaction sufficiently large to affect the highest density regions for certain dark matter candidate classes (e.g., Spergel & Steinhardt, 2000; Loeb & Weiner, 2011).

an infinite set of characteristics, which describe lines in phase space along which the distribution function is constant. For the case of the CBE characteristics equations look identical to the Newtonian equations of motion (these equations follow from the Hamiltonian equations for a Hamiltonian system):

$$\frac{d\mathbf{x}}{dt} = \mathbf{u}, \quad \frac{d\mathbf{u}}{dt} = -\nabla\Phi$$

with potential given by the Poisson equation:

$$\nabla^2\Phi = 4\pi G\rho_{\rm tot}.$$

A complete set of characteristic equations is equivalent to the solution of the CBE. N-body methods use a sub-sample of the characteristics to approximate the solution. Each sub-sample characteristics is treated as a particle moving in space under action of the collective force from all the other particles. The key ingredients of an N-body code are thus a method to calculate this collective force (gravity solver) and numerical method to integrate particle motion in space and time (particle motion integrator). Different N-body algorithms (see § 1.3 below) differ in how these two parts are implemented.

N-body approach is effectively a Monte Carlo technique for random sampling of the characteristics: i.e., a set of trajectories $\mathbf{x}(t)$, $\mathbf{v}(t)$ starting from a set of initial positions sampling distribution of matter in phase space.[3] The task of a good implementation of N-body code is to ensure that trajectories integrated by the code are faithful representations of the characteristics curves.

The number of particles and the size of simulation volume are dictated by the available computational resources and the needs of particular problem. For a correctly implemented N-body method the numerical solution is expected to approximate the true solution of the CBE on the scales of interest. The numerical solution should also converge to the full solution as $N \to \infty$. In practice, numerical implementation choices have to be made. In particular, when potential and gravitational forces are calculated, one must decide how to compute the distribution of mass. The choice that is most commonly made is to use the distribution of particles followed by N-body code and treat each particle as a localized mass with a certain extent and density profile. Recently, a different choice of reconstructing density from a special set of particles serving as mass tracers using information about initial phase space density of the dark matter sheet has also been proposed (Abel et al., 2012; Hahn et al., 2013). The choice of mass distribution reconstruction method is a gray area of N-body techniques and the optimal method likely depends on the problem at hand. Note, however, that self-consistency requires that the local density satisfies $\rho(\mathbf{x}) = \int f(t, \mathbf{x}, \mathbf{v}) d^3v$, where f is the phase space density distribution satisfying the CBE. Not every mass reconstruction method will satisfy this condition.

For many problems it is warranted to approximate evolution of baryons as collisionless and dissipationless. In this case, a mix of dark matter and baryons is treated as a single collisionless matter (i.e., baryons and dark matter are assumed to move identically) and followed using an N-body method. When collisional nature of baryons cannot be neglected, the baryons have to be treated by means of computational fluid dynamics (CFD) methods. The overall matter evolution is treated as a mixture of collisionless dark matter, followed using an N-body method, and baryons followed by a CFD method. The two components are coupled via gravity as they feel the common gravitational field and both contribute to the density in the right hand side of the Poisson equation.

1.2 Cosmological simulations: hydrodynamics

In a fully collisional particle systems with properties of an ideal gas, the Boltzmann equation can be simplified and reduced to 3D ODEs describing ideal compressible gas. This is done by taking the three first moments of the Boltzmann equations — i.e., multiplying the equation by 1, $\dot{\mathbf{x}}$, and $\dot{x}^2$ and integrating over velocity space). These moments give the continuity, Euler, and energy equations of hydrodynamics[4] (e.g., *F. Shu "Gasdynamics"*):

$$\frac{\partial\rho}{\partial t} + \nabla\rho\mathbf{u} = 0,$$

[3] Monte Carlo techniques are very often used in the otherwise intractable high-dimensional problems, like evaluating many-dimensional integrals or likelihood and parameter estimation in many-parameter problems. Markov Chain Monte Carlo (MCMC) methods are used in such problems.

[4] Equations below do not take into account additional processes commonly included in simulations: heating and cooling of gas, star formation, etc. These are included as sink and source terms in the appropriate equations. We will consider such processes on friday.

$$\frac{\partial \mathbf{u}}{\partial t} + (\mathbf{u}\cdot\nabla)\mathbf{u} = -\nabla\Phi - \frac{\nabla P}{\rho},$$
$$\frac{\partial E}{\partial t} + \nabla\cdot[(E+P)\mathbf{u}] = -\rho\mathbf{u}\cdot\nabla\Phi.$$

The equations above are closed with the equation of state, which is usually assumed to be that of an ideal gas:

$$\varepsilon = \frac{1}{\gamma-1}\frac{P}{\rho},$$

or to have some other simple form (i.e., polytropic EoS: $P = K\rho^\gamma$)

Equations of particle motion, the Poisson equation, the equations of gasdynamics above, along with the equation of state of the gas, form the set of basic equations solved in cosmological simulations.

Note that all of the equations we solve to describe the evolution of matter in the universe have their usual simple non-relativistic form. This is because most commonly the matter components relevant for late evolution move with non-relativistic velocities.

1.3 N-body codes: Historical overview

N-body simulations of structure formation evolved from $N = O(1)$ to $N = O(10^{10})$ in the last several decades. Strömgren carried out numerical integrations of a 3-body problem for a part of the orbit and noted that the method of mechanical integration can be extended to the problem of 4 bodies and beyond. The first N-body calculation related to galaxy formation, however, was performed by Holmberg (1941), who using an experimental setup with a set of 74 bulbs (representing particles), photometers and lab books to carry out a merger of two disk galaxies in two dimensions.

Direct N-body simulations in the late 1950s and 1960s, following 15-100 particles N-body, aimed to model dynamics of stellar systems and to test models of dynamical relaxation (von Hoerner, 1960; Aarseth, 1963). Direct N-body calculations were also used for early simulations of density peak collapse to model galaxy clusters (Peebles, 1970) and evolution of cosmological density fields (Fall, 1978; Aarseth et al., 1979; Efstathiou, 1979) with $N \sim 10^3$ particles. In these calculations, however, inter-particle force "softened" at small separations was used to account either for diffuse internal structure of galaxies or for collisionless nature of matter particles at the scales of interest. For a more detailed review of the history of direct N-body algorithms see Aarseth (2003) and Heggie & Hut (2003).

The computational cost of the direct N-body integration scales as $\propto N^3$ with the number of particles, which limits the method to integrations with $\lesssim 10^6$ particles on conventional computing platforms. This limit can be surpassed with the help of special purpose hardware, such as the GRAPE boards (Makino, 2008). Recently, one of the largest cosmological simulations has

Alternative approach to speed up integration on conventional computing hardware was to develop faster N-body methods. In the late 1970s-early 1980s the "particle-mass" (PM Doroshkevich et al., 1980; Klypin & Shandarin, 1983; Miller, 1983) and "particle-particle-particle-mesh" (P^3M Efstathiou & Eastwood, 1981; Efstathiou et al., 1985) algorithms, originally developed for plasma calculations, have been adopted for cosmological simulations of structure formation and used to carry out simulations with $N \sim 10^4 - 10^5$ particles. The PM algorithm uses a uniform grid to discretize density and solve for the potential using the Fast Fourier Transform (this algorithm is discussed in detail in Chapter 3 below). The particles move through the grid using forces calculated from potential on the grid. As particles move, the density in grid cells is updated by interpolating from particle positions. Particle movement is an operation that scales linearly with particle number $\propto N$, while FFT scales as $\propto N_c \log N_c$ with the number of grid cells N_c. The P^3M algorithm uses the PM scheme and adds calculation of small-scale forces by direct summation over particles in neighboring grid cells. In this algorithm small-scale force is combined with the large-scale force calculated from the PM potential (see Hockney & Eastwood, 1981, for detailed description of the PM and P^3M algorithms). The particle-particle part of the algorithm can become expensive when a significant fraction of particles collapses into dense small clumps (e.g., late stages of gravitational collapse in structure formation models). This issue has been ameliorated by introduction of additional computational grids to reduce the fraction of particles for which particle-particle-forces are computed (Couchman, 1991), which leads to a factor of $\sim 10 - 20$ speed up in computing times for particle distributions.

In the mid-1980s, gridless "tree" algorithm was developed, which uses multipole expansion to calculate gravitational forces (Barnes & Hut, 1986). This algorithm does not impose a grid on the computational domain, but constructs an hierarchical tree using particle positions and uses the tree structure for efficient

computation of multipole expansion terms. The computational cost scales as $\propto N \log N$ with the number of particles – a great improvement over the direct summation codes, which comes at a cost of controllable force error. The Tree algorithm is particularly advantageous compared to grid codes in situations that do not require periodic boundary conditions. They are also not constrained by the rectangular geometry of grid or the limits on the number of grid cells. Nevertheless, the Tree algorithm may perform poorly during early stages of structure evolution, when matter distribution is close to uniform and, overall, it is more computationally expensive than PM or P^3M (and especially AP^3M) algorithms. Therefore, hybrid TreePM algorithm was developed in the 1990s (Xu, 1995; Bode et al., 2000), in which the large-scale force computed using the FFT method on a base cubic grid is combined with the small-scale force computed by the tree algorithm.

In the 1990s, additional approximate N-body algorithms have been developed. In one class of methods, several grids are adaptively introduced to increase spatial resolution locally. Villumsen (1989) has developed an hierarchical PM method, in which in addition to the base cubic grid an hierarchy of additional cubic subgrids is introduced. The potential on subgrids is computed using FFT method with isolated boundary conditions and the force on a particle is computed as a sum of forces from all of the parent grids. The placement of grids in this method is not adaptive and so it achieves high dynamic range only in a given cubic sub-volume within the computational domain. AP^3M method of Couchman (1991) introduces additional rectangular grids adaptively to speed up the particle-particle part of the P^3M calculations. The total force on each particle is now computed as a sum of the direct force from neighbors and a sum of forces from parent grids, where the forces are computed using FFT method. The forces are shaped so that they add up to the total softened Newtonian force. A similar method is used to compute forces in the Enzo adaptive mesh refinement (AMR) code (Bryan & Norman, 1997).

Alternative approach is to use adaptively introduced finer meshes and solve for potential on these meshes using multigrid approach (Suisalu & Saar, 1995). Kravtsov et al. (1997) introduced a hybrid approach, in which FFT method is used to solve the Poisson equation on the base cubic grid that covers the entire computational domain, while on adaptively introduced refinement meshes the potential is obtained via successive over-relaxation method using solution interpolated from the previous level as initial guess and as a boundary condition (see also Teyssier, 2002). Additional schemes that use moving mesh instead of adaptive refinements have also been developed (Gnedin, 1995; Pen, 1998), although practice has shown that significant restrictions need to be put on the grid motion and distorsions in order to avoid large numerical errors (Gnedin & Bertschinger, 1996). Review of the methods developed up until late 1990s can be found in Bertschinger (1998).

Since the late 1990s, the new implementations of N-body codes have mostly featured refinements and optimizations of the previously developed algorithms described above. One exception is a new approach presented by Abel et al. (2012) and Hahn et al. (2013) who propose a different way to model mass distribution of matter in cosmological structure formation simulations, separating the flow tracers representing characteristics of the Boltzmann equation and mass tracers. This approach improves density representation in the low density regions of simulations, which is particularly important to minimize artificial fragmentation of filaments in Warm Dark Matter models (Wang & White, 2007). Substantial effort has been put into developing parallel versions of the algorithms suitable for rapidly increasing number of processors on massively parallel platforms.

We will use the PM method to introduce the key concepts and issues of N-body simulations. By developing your own PM code you will get hands on experience in actually building a full N-body code. Although PM scheme is relatively simple, PM code does include all main components of any cosmological N-body code. Using your own PM code you will run tests and cosmological simulations and will explore numerical issues. You will also explore limitations of the PM scheme.

2.4 Particle-Mesh (PM) technique: a brief history

- PM was invented in the 1950's at LANL for simulations of compressible fluid flows.
- The first wide-spread application was for collisionless plasma simulations (for which PM was reinvented in the 1960s and popularized by Hockney and collaborators)
- In the late 1970s PM was first applied for 3D cosmological simulations. Although the pure PM method is no longer used in cosmological simulations due to its resolution limitations, the method is now used as a key ingredients in most commonly used cosmological codes (P^3M, AP^3M, ART and RAMSES, TreePM, etc.)
- The popularity of the PM method in cosmology is due to
  - its relative algorithmic simplicity and speed (the running time scales as $\propto O(N_p) + O(N_c \ln N_c)$, where N_p is the number of particles and N_c is the number of grid cells;
  - natural incorporation of periodic boundary conditions;
- Although PM is no longer used in pure form for cosmological simulations, it remains an important ingredient in most widely used cosmological codes (TreePM codes, P^3M, and AP^3M, ART, Enzo, etc.).

2.5 Model equations

PM code solves the Poisson equation

$$\nabla^2\Phi = 4\pi G\rho_{\rm tot} - \Lambda,$$

where Φ is the proper gravitational potential, $\rho_{\rm tot}$ is the total matter density and Λ is the cosmological constant term, and equations of motion of particles

$$\frac{d\mathbf{r}}{dt} = \mathbf{u}; \quad \frac{d\mathbf{u}}{dt} = -\nabla\Phi,$$

note that all variables are defined in proper coordinates and all spatial derivatives are also taken with respect to these coordinates.

However, for modelling of matter evolution in expanding universe it is convenient to re-write the equations in *comoving* variables and make them dimensionless by choosing suitable units (We will denote variables in dimensionless code units with a tilde ˜). In the following, we will adopt the variables and units used in Anatoly Klypin's PM code. This is just an example. Feel free to choose your own - just make sure you are consistent!

$$\tilde{\mathbf{x}} \equiv a^{-1}\frac{\mathbf{r}}{r_0}, \quad \tilde{\mathbf{p}} \equiv a\ \frac{\mathbf{v}}{v_0}, \quad \tilde{\phi} \equiv \frac{\phi}{\phi_0}, \quad \tilde{\rho} = a^3\frac{\rho}{\rho_0},$$

where $\mathbf{x}$ is the comoving coordinates, $\mathbf{v} = \mathbf{u} - H\mathbf{r} = a\dot{\mathbf{x}}$ is the *peculiar velocity*[5] and ϕ is the *peculiar potential* defined as (Peebles 1980, p. 42)

$$\phi \quad = \quad \Phi + 1/2\ a\ddot{a}\,(r/a)^2 = \Phi + \frac{H^2}{2}\left(\Omega_\Lambda - \frac{1}{2}a^{-3}\Omega_\mathrm{m}\right) r^2,$$

where

$$\Omega_\mathrm{m} = \frac{8\pi G\bar{\rho}}{3H^2}; \quad \Omega_\Lambda = \frac{\Lambda}{3H^2}.$$

Here Ω_m, Ω_Λ, and H are mean densities of matter and cosmological constant in units of critical density, and H is the Hubble constant; all these quantities are at $z = 0$.

The variables with subscript zero are the units in which corresponding physical variables are measured. The unit of length, r_0, is an arbitrary scale. Let us choose r_0 to be the size of the PM grid cell:

$$r_0 = \frac{L_\mathrm{box}}{N_g}; \quad N_g^3 = \text{total number of grid cells}$$

and the unit of time to be inverse Hubble constant (the Hubble constant):

$$t_0 \equiv H_0^{-1},$$

the rest of the units are then defined as

$$\begin{aligned} v_0 &\equiv \frac{r_0}{t_0}, \\ \rho_0 &\equiv \frac{3H_0^2}{8\pi G}\Omega_{m,0}, \\ \phi_0 &\equiv \frac{r_0^2}{t_0^2} = v_0^2. \end{aligned}$$

It is also convenient to choose the expansion factor as time variable (using expressions for the Hubble constant: $\dot{a} = aH(a)$). For this choice of variables, the Poisson equation and equations of motion can be re-written as

$$\nabla^2\phi = 4\pi G\Omega_{m,0}\rho_{crit,0}a^{-1}\delta, \quad \delta = \frac{\rho - \bar{\rho}}{\bar{\rho}},$$

$$\frac{d\mathbf{p}}{da} = -\frac{\nabla\phi}{\dot{a}}, \quad \frac{d\mathbf{x}}{da} = \frac{\mathbf{p}}{\dot{a}a^2}.$$

where δ is the overdensity in comoving coordinates and $\dot{a}$ is

$$\dot{a} = H_0 a^{-1/2}\sqrt{\Omega_{m,0} + \Omega_{k,0}a + \Omega_{\Lambda,0}a^3}; \quad \Omega_{m,0} + \Omega_{\Lambda,0} + \Omega_{k,0} = 1.$$

In dimensionless variables the equations are:

$$\tilde{\nabla}^2\tilde{\phi} = \frac{3}{2}\frac{\Omega_0}{a}\tilde{\delta},$$

$$\frac{d\tilde{\mathbf{p}}}{da} = -f(a)\tilde{\nabla}\tilde{\phi}, \quad \frac{d\tilde{\mathbf{x}}}{da} = f(a)\frac{\tilde{\mathbf{p}}}{a^2}.$$

where $\tilde{\delta} = \tilde{\rho} - 1$ and

$$f(a) \equiv H_0/\dot{a} = \left[a^{-1}\left(\Omega_{m,0} + \Omega_{k,0}a + \Omega_{\Lambda,0}a^3\right)\right]^{-1/2}.$$

These equations are used in the three main steps of a PM code:

- Solve the Poisson equation using the density field estimated with current particle positions.
- Advance momenta using the new potential.
- Update particle positions using new momenta.

[5] $p \propto av$ is called momentum, this choice of variable allows us to get rid of the annoying $(\dot{a}/a)$ terms in equations.

2.6 Data structures

For a PM code with the second-order accurate time integration we need the minimum of *six real numbers for positions and momenta for each particle* (assuming particles have the same mass) and *one real number for the potential of each grid cell.*

The array for the potential can be shared between density and potential: first use it for density, then replace it with potential when the Poisson equation is solved. However, for simplicity you can start with two grid arrays (for density and potential). Also, you will probably need auxiliary arrays for FFT, depending on which FFT solver you choose to use.

The convenient data structures are 1D or 3D arrays for particles (e.g., six 1D arrays for `x(i), y(i), z(i), vx(i), vy(i), vz(i)`) and 3D arrays for the grid variables (e.g., `rho(i,j,k), phi(i,j,k)`). When up loops with iterations over arrays you should be mindful of how arrays are stored in memory by a particular language you use (which array index corresponds to the nearby locations in memory). In addition, array accesses can be arranged to minimize cache misses. Modern computers employ memory hierarchy with smaller amounts of memory in fast caches. Compilers try to predict memory access pattern and bring in data into caches before it is accessed. You can help it by arranging memory accesses in an ordered manner. Cache misses result in a direct access of main RAM, which is orders of magnitude more expensive than accessing data in caches. Minimizing cache misses can substantially improve performance of a code.

Run your first tests for (32^3, 64^3) or (64^3, 128^3) particles and cells in which case memory requirements should not be an issue.

2.7 Density Assignment

The Particle Mesh algorithm assumea that particles have certain size, mass, shape, and internal density. This determines the interpolation scheme used to assign densities to grid cells. Let's define the 1D particle shape, $S(x)$, to be mass density at the distance x from the particle for cell size Δx (Hockney & Eastwood 1981). The common choices are

- Nearest Grid Point (NGP): particles are point-like and all of particle's mass is assigned to the single grid cell that contains it:
$$S(x) = \frac{1}{\Delta x}\delta\left(\frac{x}{\Delta x}\right)$$
- Cloud In Cell (CIC): particles are cubes (in 3D) of uniform density and of one grid cell size.
$$S(x) = \frac{1}{\Delta x}\begin{cases} 1, & |x| < \frac{1}{2}\Delta x \\ 0, & \text{otherwise} \end{cases}$$
- Triangular Shaped Cloud (TSC):
$$S(x) = \frac{1}{\Delta x}\begin{cases} 1 - |x|/\Delta x, & |x| < \Delta x \\ 0, & \text{otherwise} \end{cases}$$

The fraction of particle's mass assigned to a cell ijk is the shape function averaged over this cell:

$$W(x_p - x_{ijk}) = \int\limits_{x_{ijk}-\Delta x/2}^{x_{ijk}+\Delta x/2} dx' S(x_p - x');$$

$$W(\mathbf{r}_p - \mathbf{r}_{ijk}) = W(x_p - x_{ijk})W(y_p - y_{ijk})W(z_p - z_{ijk});$$

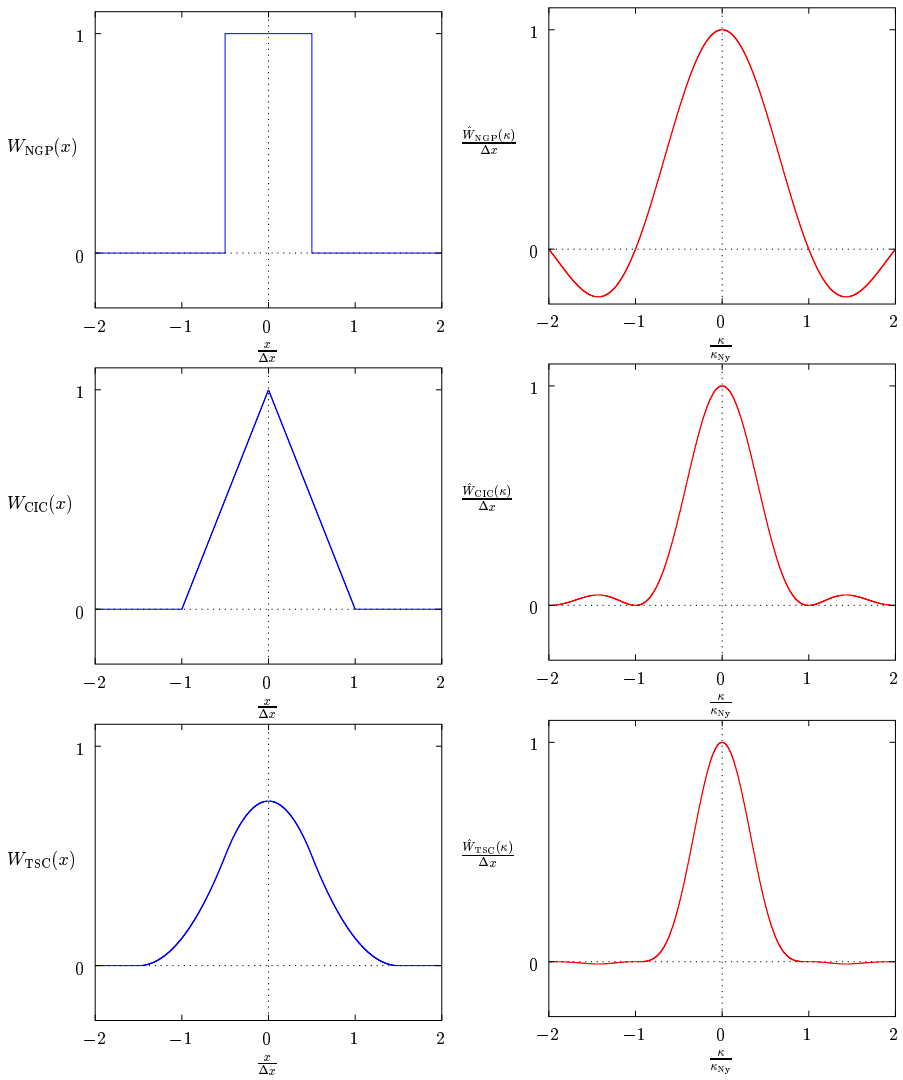
The density in a cell ijk is then

$$\rho_{ijk} = \sum_{p=1}^{N_p} m_p W(\mathbf{r}_p - \mathbf{r}_{ijk})$$

In practice, of course, we loop over particles and assign the density to neighboring cells as opposed to summing over all particles for each cell as the straightforward reading of the above equation would suggest.

3-9

the density assignment wider in configuration space.



PM density interpolation kernels in real and Fourier space (figure by M. Gross)

2.8 Implementing the CIC interpolation

The choice of interpolation scheme is a tradeoff between accuracy and computational expense. The CIC scheme is both relatively cheap and accurate and is most commonly used in PM codes. I will now describe

how to implement it (you can start coding with the simpler NGP scheme and upgrade it to CIC later when your code is tested).

Consider the density assignment for a particle with coordinates $\{x_p, y_p, z_p\}$. The cell containing the particle will have indices of

$$i = [x_p]; \quad j = [y_p]; \quad k = [z_p],$$

where $[x]$ is the integer floor function (equivalent to Fortran's `int`). Let's assume that cell centers are at $\{x_c, y_c, z_c\} = \{i, j, k\}$ (note that in internal units cell size $\Delta x = 1$, this will be implicitly assumed below). This is a matter of convention, but once chosen the convention should be consistent throughout the code.

In the CIC scheme, particle $\{x_p, y_p, z_p\}$ may contribute to densities in the parent cell (i, j, k) and seven neighboring cells[6]. Let's define

$$d_x = x_p - x_c; \quad d_y = y_p - y_c; \quad d_z = z_p - z_c;$$

$$t_x = 1 - d_x; \quad t_y = 1 - d_y; \quad t_z = 1 - d_z.$$

Contributions to the eight cells are then linear interpolations in 3D:

$$\begin{aligned}
\rho_{i,j,k} &= \rho_{i,j,k} + m_p t_x t_y t_z; \quad \rho_{i+1,j,k} = \rho_{i+1,j,k} + m_p d_x t_y t_z; \\
\rho_{i,j+1,k} &= \rho_{i,j+1,k} + m_p t_x d_y t_z; \quad \rho_{i+1,j+1,k} = \rho_{i+1,j+1,k} + m_p d_x d_y t_z; \\
\rho_{i,j,k+1} &= \rho_{i,j,k+1} + m_p t_x t_y d_z; \quad \rho_{i+1,j,k+1} = \rho_{i+1,j,k+1} + m_p d_x t_y d_z; \\
\rho_{i,j+1,k+1} &= \rho_{i,j+1,k+1} + m_p t_x d_y d_z; \quad \rho_{i+1,j+1,k+1} = \rho_{i+1,j+1,k+1} + m_p d_x d_y d_z;,
\end{aligned}$$

where m_p is particle mass. Doing this for all particles will result in the grid of densities $\rho_{i,j,k}$.

2.9 Solving the Poisson equation

With the grid of densities, $\rho_{i,j,k}$, in hand, the code can proceed to solve the discretized[7] Poisson equation

$$\begin{aligned}
\tilde{\nabla}^2 \tilde{\phi} &\approx \tilde{\phi}_{i-1,j,k} + \tilde{\phi}_{i+1,j,k} + \tilde{\phi}_{i,j-1,k} + \tilde{\phi}_{i,j+1,k} + \tilde{\phi}_{i,j,k-1} + \tilde{\phi}_{i,j,k+1} - 6\tilde{\phi}_{i,j,k} = \\
&= \frac{3}{2}\frac{\Omega_0}{a}(\tilde{\rho}_{i,j,k} - 1).
\end{aligned}$$

The discretization thus results in a large system of linear equations relating unknowns, $\tilde{\phi}_{i,j,k}$, to the known right hand side values. This system can be solved using FFT.

In the Fourier space the Poisson equation is $\tilde{\phi}(\mathbf{k}) = G(\mathbf{k})\tilde{\delta}(\mathbf{k})$, where $G(\mathbf{k})$ is the Green function which for the adopted discretization is given by

$$G(\mathbf{k}) = -\frac{3\Omega_0}{8a}\left[\sin^2\left(\frac{k_x}{2}\right) + \sin^2\left(\frac{k_y}{2}\right) + \sin^2\left(\frac{k_z}{2}\right)\right]^{-1},$$

where $L = N_g$ is the box size in code units and

$$k_x = 2\pi l/L, \quad k_y = 2\pi m/L, \quad k_z = 2\pi n/L, \quad \text{for component } (l, m, n).$$

The singularity at $l = m = n = 0$ should be avoided by setting this component of potential, $\hat{\phi}_{000}$, by hand to zero.

$\Rightarrow$ Given a density field $\tilde{\delta}(\mathbf{r})$ in real space, we can solve for the gravitational potential by

- performing the FFT to get $\tilde{\delta}(\mathbf{k})$,
- multiplying every element in the field by the corresponding value of $G(\mathbf{k})$ to get $\tilde{\phi}(\mathbf{k})$,

$$\hat{\phi}_{lmn} = G(\mathbf{k}_{lmn})\hat{\delta}_{lmn}, \text{where}$$

$$\hat{f}_{lmn} = (\Delta x)^3 \sum_{i,j,k=0}^{N_g-1} f_{ijk} e^{-i2\pi(il+jm+kn)/N_g},$$

$$f_{ijk} = \frac{1}{L^3} \sum_{l,m,n=0}^{N_g-1} \hat{f}_{lmn} e^{i2\pi(il+jm+kn)/N_g}.$$

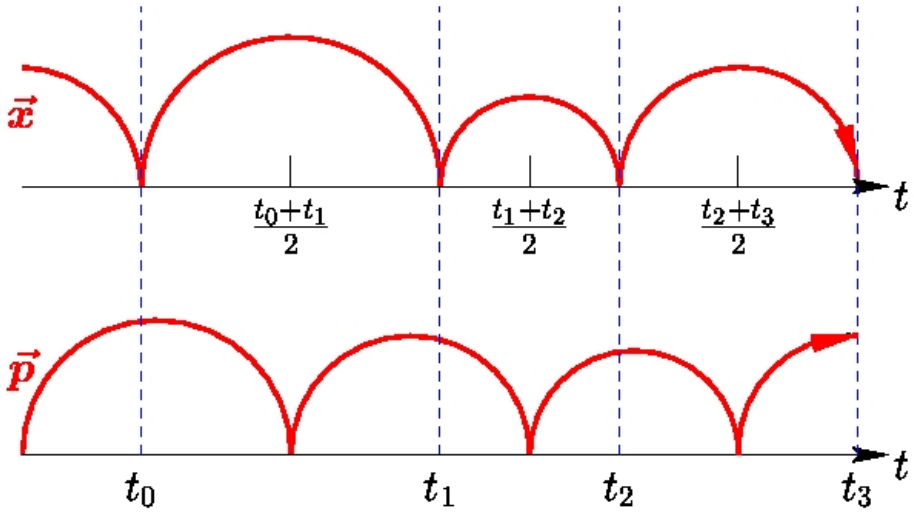
[6] Make sure you enforce periodic boundary conditions: $i = \mathrm{mod}(i, N_{g,1})$, etc. for j and k.

[7] It is customary in PM codes to discretize the Laplacian operator using the 7-point template.

- transforming[8] the result back to real space to get $\tilde{\phi}(\mathbf{r})$ discretized at cell centers.

2.10 Updating particle positions and velocities

Let us assume we are using constant integration step in Δa and the second-order accurate *leapfrog* integration.



Schematic of the leapfrog integration for a variable time step.
(adopted from M.Gross's PM notes).

After n time steps, $a_n = a_i + n\Delta a$, during the step $n+1$, we should have coordinates $\tilde{\mathbf{x}}_n$ at a_n and momenta $\tilde{\mathbf{p}}_{n-1/2}$ at $a_{n-1/2} = a_n - \Delta a/2$ from the previous step. Assigning density and solving the Poisson equation gives potential $\tilde{\phi}_n$ at a_n. For the assumed variables and units, positions and momenta are updated as follows:

$$\tilde{\mathbf{p}}_{n+1/2} = \tilde{\mathbf{p}}_{n-1/2} + f(a_n)\tilde{\mathbf{g}}_n\Delta a; \quad \tilde{\mathbf{x}}_{n+1} = \tilde{\mathbf{x}}_n + a_{n+1/2}^{-2} f(a_{n+1/2})\tilde{\mathbf{p}}_{n+1/2}\Delta a.$$

Here, $\tilde{\mathbf{g}}_n = -\tilde{\nabla}\tilde{\phi}_n$ is acceleration *at the particle's position*. This acceleration can be obtained by interpolating accelerations from the neighboring cell centers. The latter are given by

$$\tilde{g}^x_{i,j,k} = -(\tilde{\phi}_{i+1,j,k} - \tilde{\phi}_{i-1,j,k})/2, \quad \tilde{g}^y_{i,j,k} = -(\tilde{\phi}_{i,j+1,k} - \tilde{\phi}_{i,j-1,k})/2,$$

$$\tilde{g}^z_{i,j,k} = -(\tilde{\phi}_{i,j,k+1} - \tilde{\phi}_{i,j,k-1})/2.$$

Note that these equations are in dimensionless code units. In these units, cell size $\Delta x = 1$, so the cell size in the denominators are implicit.

For each component of the acceleration, you should use the same interpolation scheme as during density assignment. For a given particle the acceleration should be interpolated from the cells to which particle contributed density during density assignment step. **This is important!** Consistency in interpolation schemes ensures absence of artificial self-forces and the third Newton's law (Hockney & Eastwood 1981).

For the NGP scheme, acceleration for a particle is just the acceleration of its parent cell $\tilde{\mathbf{g}}_{i,j,k}$. For the CIC interpolation, we do the following (indices (i,j,k) here correspond to the parent cell of the particle; see section on density assignment):

$$\begin{aligned} g^x_p \quad = \quad & g^x_{i,j,k}t_x t_y t_z + g^x_{i+1,j,k}d_x t_y t_z + g^x_{i,j+1,k}t_x d_y t_z + g^x_{i+1,j+1,k}d_x d_y t_z + \\ & g^x_{i,j,k+1}t_x t_y d_z + g^x_{i+1,j,k+1}d_x t_y d_z + g^x_{i,j+1,k+1}t_x d_y d_z + g^x_{i+1,j+1,k+1}d_x d_y d_z. \end{aligned}$$

And the same for g^p_y and g^p_z. $t_{x,y,z}$ and $d_{x,y,z}$ have the same definition as in the CIC density assignment described above.

After updating particle positions and velocities, the code should do some useful I/O and then proceed to step $n+2$.

[8] Be careful about normalization of the FFT. The transforms $\tilde{\delta}(\mathbf{r}) \to \tilde{\delta}(\mathbf{k})$ and $\tilde{\delta}(\mathbf{k}) \to \tilde{\delta}(\mathbf{r})$ should recover the original field $\tilde{\delta}(\mathbf{r})$.

2.11 Zel'dovich approximation

For the first basic test of your PM code you can test evolution of a simple sine wave against evolution predicted by the Zel'dovich approximation, which is actually exact for a planar wave. Here is basic info about the Zel'dovich approximation.

- First-order (linear) Lagrangian collapse model
  [Zeldovich 1970; see also Peacock's (p. 485) and Padmanabhan's (p. 294) textbooks for pedagogical introduction]
- Zeldovich approximation can be expressed by one equation:

$$\mathbf{x}(t) = \mathbf{q} + D_+(t)\mathbf{S}(\mathbf{q}),$$

  it may look more familiar if you think about it as

$$\mathbf{x}(t) = \mathbf{x_0} + \mathbf{v}t; \quad \mathbf{v} = const$$

  where

$\mathbf{q}$	is the initial position of a matter parcel (a particle in N-body simulations) and
$\mathbf{x}(t)$	is its position at time t (both $\mathbf{q}$ and $\mathbf{x}$ are *comoving*).
$D_+(t)$	is the linear growth function [see eq. (29) in Carrol et al. (1993) for a useful fitting formula] and
$\mathbf{S}(\mathbf{q})$	is a *time-independent* displacement vector.

- The evolution of density follows from conservation of mass $\rho(\mathbf{x})d^3\mathbf{x} = \bar{\rho}d^3\mathbf{q}$:

$$\rho(\mathbf{x}, t) = \frac{\bar{\rho}}{\det(\partial x_j/\partial q_i)} = \frac{\bar{\rho}}{\det[\delta_{ij} + D_+(t) \times (\partial S_j/\partial q_i)]}$$

- ZA is exact in 1D until the first crossing of particle trajectories occurs. This is because in 1D ZA describes collapse of parallel sheets of matter and acceleration of each sheet is independent of $\mathbf{x}$ until sheets cross.
- Although ZA is only an approximation in 3D, it still works very well at the initial stages of evolution of cosmological density fields because flattened *pancake* structures (whose evolution is well described by ZA) are typical in such fields[9] [see, e.g., Bond et al. (1996)]. This has made ZA a popular choice for setting up initial conditions in cosmological simulations (Klypin & Shandarin, 1983; Efstathiou et al., 1985). More recently, more accurate second-order Lagrangian perturbation theory (2LPT) has also been used to improve accuracy of initial conditions (see Crocce et al., 2006).

2.12 A test problem: 1D collapse of a plane wave

Let's consider evolution of a 1D sine-wave $S(q) = A\sin(kq)$:

$$x(a) = q + D_+(a)A\sin(kq); \quad k = 2\pi/\lambda;$$

Let's assume we want to simulate a wave with wavelength equal to the simulation box size, $\lambda = L_{\rm box} = N_g$ using $N_{p,1}^3$ particles and N_g^3 grid cells in a cubic grid. In 3D, the 1D equations can be used to set up initial conditions for $N_{p,1}$ parallel sheets of particles. For some initial epoch, $a_{\rm ini}$, we have

$$x_i(a_{\rm ini}) = q_i + D_+(a_{\rm ini})A\sin\left(\frac{2\pi q_i}{L_{\rm box}}\right); \quad q_i = (i-1)\frac{L_{\rm box}}{N_{p,1}} \text{ for } i = 1, ..., N_{p,1};$$

Take the time derivative of x to get equation for *peculiar* velocity $v = a\dot{x}$ (or momentum $p = av$, if needed):

$$v(x) = a\dot{D}_+(a_{\rm ini} - \Delta a/2)A\sin(kq),$$

[9] Another explanation of the success of ZA is its use of displacement field, $S(q)$, as the basis for evolution model. Density depends on the derivatives of $S(q)$ so that small (linear) displacements can correspond to large density contrasts.

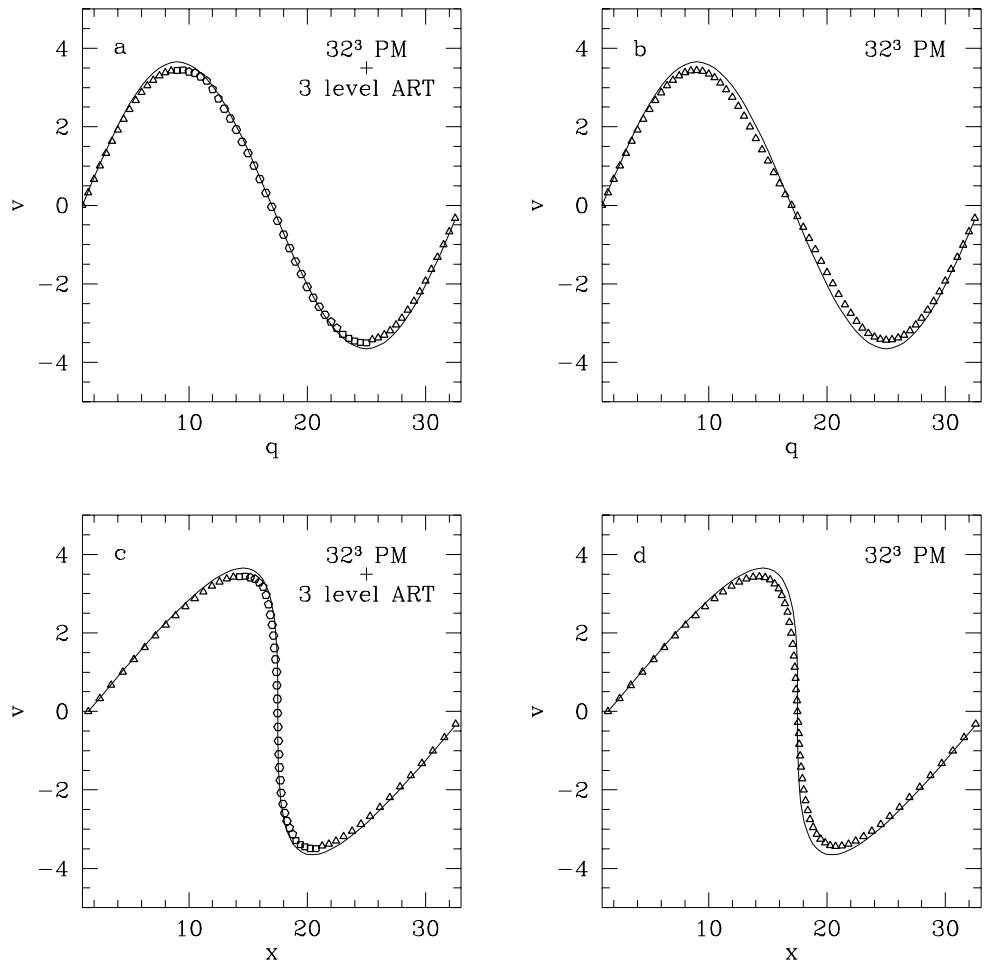


Figure 2.1: Plane wave collapse test: phase diagram in Lagrangian coordinates q (a) the ART simulation with 32^3 base grid and 3 refinement levels and (b) the PM simulation with a 32^3-cell grid at the crossing time. *Solid line*, analytic solution; *polygons*, numerical results.(c,d) Corresponding phase diagram for physical coordinates x.

where initialization for the leapfrog scheme is assumed. We can choose the first crossing epoch, say $a_{\rm cross} = 10a_{\rm ini}$. The epoch of the first crossing can then be identified as the epoch of the caustic formation (i.e., $\rho(x_{\rm cross}, a_{\rm cross}) = \infty$, where $x_{\rm cross} = q_{\rm cross} = L_{\rm box}/2$), which gives us the wave amplitude A:

$$\rho(x,a) = \frac{\bar{\rho}}{[1 + D_+(a) \times Ak\cos(kq)]}, \quad \Rightarrow A = -\left(D_+(a_{\rm cross})k\right)^{-1}.$$

Positions $x(q,a)$ and velocities $v(q,a)$ is all we need to set up initial conditions for particles.

The subsequent 1D evolution using the PM code can be tested by comparing x and v given by the above equations to the coordinates and velocities of particles at different epochs using outputs of your code (for example, you can plot phase diagrams, i.e. particles in $v-x$ or $v-q$ plane).

In addition, you can use solutions for the gravitational potential and particle accelerations to test your gravity solver. For each grid cell:

$$\phi(x,a) = \frac{3}{2}\frac{\Omega_0}{a}\left\{\frac{q^2-x^2}{2} + D_+ \times \frac{A}{k}\left[kq\sin(kq) + \cos(kq) - 1\right]\right\};$$

$$g(x,a) = -\frac{\partial\phi}{\partial x} = \frac{3}{2}\frac{\Omega_0}{a}(x-q) = \frac{3}{2}\frac{\Omega_0}{a}D_+(a)A\sin(kq)$$

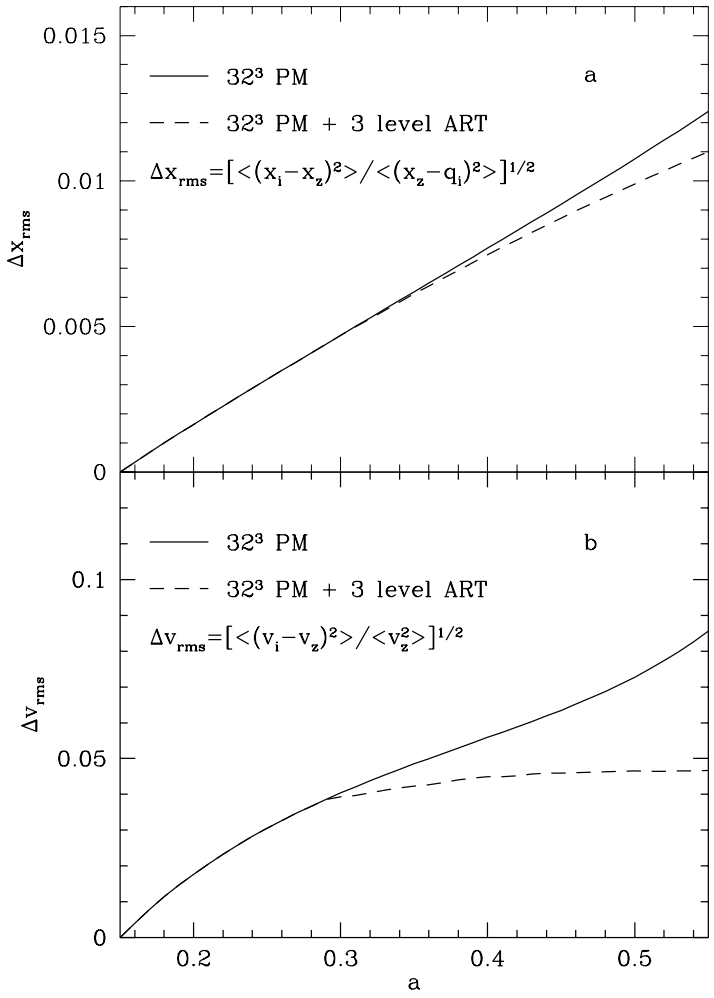


Figure 2.2: Plane wave collapse test: rms deviations of (a) coordinates and (b) velocities from analytic solution vs. the expansion parameter for the PM code (*solid line*) and for the ART code with three levels of refinement (*dashed line*) See, Efstathiou et al. (1985) for details on this and other tests.

The only difference from particles is that you do not know cell's Lagrangian coordinate q (q is known for particles if you maintain particles in the same order throughout evolution; particle's index gives q as we saw above). For a given expansion factor a, q of a cell can be calculated by solving equation $q = x - D_+(a)A\sin(kq)$ numerically, with x is coordinate of the cell.

2.13 Beyond the Zel'dovich test: setting up cosmological initial conditions

If your code passes the Zeldovich test, you may try to run a realistic cosmological simulation. To do this, you will have to code a routine to set up initial conditions for the particles using a statistical realization of the power spectrum, $P(k)$, of your favourite cosmological model. Fortunately, the basis for the algorithm is the now familiar ZA and we will discuss how to do this in detail in the next chapter. The main idea is that displacement of a particle is now determined not by a single wave, but by the entire set of waves that can be represented numerically in the simulation box. Thus, particle's comoving coordinates and momenta, $p = a^2\dot{x}$, are given by

$$\mathbf{x} = \mathbf{q} - D_+(a)\mathbf{S}(\mathbf{q}); \quad \mathbf{p} = -(a - \Delta a/2)^2\dot{D}_+(a - \Delta a/2)\mathbf{S}(\mathbf{q}),$$

where a is the initial expansion factor and Δa is its step[10], $\mathbf{q}$ is particle's unperturbed position.

The *displacement vector* $\mathbf{S}$ is given by the discrete Fourier transform (e.g., Padmanabhan 1993, p.294):

$$\mathbf{S}(\mathbf{q}) = \alpha \sum_{k_{x,y,z}=-k_{max}}^{k_{max}} i\mathbf{k}\, c_k \exp\left(i\mathbf{k}\cdot\mathbf{q}\right);$$

$$k_{x,y,z} = \frac{2\pi}{N_g}\, l,m,n; \quad l,m,n = 0,\pm 1,...,\pm N_{p,1}/2; \quad k^2 = k_x^2 + k_y^2 + k_z^2 \neq 0.$$

Here, α is the power spectrum normalization. The summation is over all possible wavenumbers from the fundamental mode with wavelentgh equal to the box size to the smallest "Nyquist" wavelength with the wavenumber of $N_{p,1}/2$.

Thus, if we construct a realization of $\{k_{x,y,z}c_k\}$ using a cosmological power spectrum on a grid in the Fourier space, its discrete FFT gives us components of the real space displacement vector $\mathbf{S}(\mathbf{q})$ for all particles. Applying these equations in practice for N_p particles on a cubic grid with N_g^3 cells amounts to

1) distributing particles uniformly in the simulation volume (e.g., placing particles in the centers of appropriately spaced grid cells).

2) Computing the displacement vector for each particle position. Construct a realization of $k_{x,y,z}c_k$ on three (each for one component of the wavevector) cubic grids. For example, for the x-component, the grid is initialized to $k_x c_k$ where k_x are running from $-k_{Ny}$ to k_{Ny} (assume $k_{Ny} = N_{p,1}/2$, k here is in units of the fundamental mode $2\pi/L_{box} = 2\pi/N_g^{1/3}$) and a_k and b_k are gaussian random numbers defined above.

3) FFT each of the three grids to get $S_x(q)$, $S_y(q)$, and $S_z(q)$ in real space. Apply ZA to displace particles from their Lagrangian positions ($\mathbf{q} \to \mathbf{x}$).

You will need

- Your favorite FFT solver. We recommend using the widely available FFTW library.
- A decent random number generator. Be careful here as you will need to generate fairly long sequences of random numbers. The Gaussian random pairs can be generated from the uniformly distributed numbers using the Box-Muller method (*Numerical Recipes, Ch. 7*).
- A routine returning $P(k)$ for a specific cosmological model. See Hu & Sugiyama 1996 and Eisenstein & Hu 1999 for useful analytical approximations to $P(k)$. For highly accurate results, power spectrum generated with a Boltzmann code should be used.[11]

You can start with a simulation of $\Omega_0 = 1$ CDM universe. Evolution in this model is simple: $D_+(a) = (a/a_0)$. You can check D_+ by computing the two-point correlation function of DM particles at different epochs or computing the power spectrum.

2.14 PM in cosmology: some historical references

- Efstathiou G. & Eastwood J. 1981, MNRAS **194**, 503
- Klypin A.A. & Shandarin S.F. 1983, MNRAS **204**, 891
- Centrella J. & Melott A.L. 1983, Nature **305**, 196-198
- Miller R.H., 1983, ApJ **270**, 390-409
- Efstathiou G., Davis M., White S.D.M., & Frenk C.S. 1985, ApJS **57**, 241-260

[10] Note that the growth factor $D_+(a)$ is usually scale-independent (e.g., for all models with CDM only) and this is assumed here. This is not true, however, for some models, such as the Cold+Hot Dark Matter (CHDM).

[11] One example is CAMB: `camb.info`

2.15 Books and papers with useful info on PM

- Hockney, R. W., and Eastwood, J. W. 1981, *"Computer Simulation Using Particles"*, McGraw-Hill, New York
- Klypin A.A. & Shandarin S.F. 1983, MNRAS **204**, 891
- Efstathiou G., Davis M., White S.D.M., & Frenk C.S. 1985, ApJS **57**, 241-260
  [Review and comparison of cosmological N-body methods]
- Sellwood J.A. 1987, ARA&A **25**, 151
  [Review of particle simulation methods]
- Description of Hugh Couchman's AP^3M code
- Bertschinger E. 1998, ARA&A **36**, 599
  [The most recent review of numerical techniques used in cosmological simulations and algorithms for setting up the initial conditions.]
- Klypin, A. & Holtzman, J. 1997, `astro-ph/9712217`
  "Particle-Mesh code for cosmological simulations"

Generating Initial Conditions

The starting point for a cosmological simulation is the choice of background cosmological model, the properties of the matter contents of the Universe, and properties of inhomogeneities in the matter distribution at the initial time when the simulation is started. These assumptions define the statistical properties of density fluctuations and their power spectrum (and higher-order statistics for the non-Gaussian spectra). In this chapter, we address the specific algorithms of creating realizations of cosmological density fields with specified properties. We will discuss the statistical properties of Gaussian initial conditions and their power spectra for different cosmological models in the next chapter.

3.16 Random realizations of density field

3.16.1 Random realizations of the density field in a linear regime

Let us first consider an "average", random volume of the universe, such that the density field can be considered statistically homogeneous and isotropic. This means that correlation function is a function of separation only,

$$\langle \delta(\vec{x}_1)\delta(\vec{x}_2)\rangle = \xi(|\vec{x}_1 - \vec{x}_2|).$$

For such field, a particular realization of the density distribution in space can be represented as Fourier transform of a square root of the power spectrum, modified by uncorrelated random amplitudes,

$$\delta(\vec{x}) = \frac{1}{(2\pi)^3}\int d^3k \sqrt{P(k)}\lambda_{\vec{k}} e^{\mathrm{i}\vec{k}\vec{x}}, \tag{3.1}$$

where $\lambda_{\vec{k}}$ are complex random numbers, satisfying the following *independence condition*:

$$\left\langle \lambda_{\vec{k}_1}\lambda^*_{\vec{k}_2}\right\rangle = (2\pi)^3\delta^3_D(\vec{k}_2 - \vec{k}_1). \tag{3.2}$$

Here δ^3_D is the three-dimensional Dirac delta-function[12]. Note that amplitudes[13] $\lambda_{\vec{k}}$ are always uncorrelated for a statistically homogeneous and isotropic field, even if the density field is non-Gaussian.

Given that the density field in the real space $\vec{x}$ has to be represented by real, rather than complex, numbers, the amplitudes $\lambda_{\vec{k}}$ must also satisfy the Hermitian condition,

$$\lambda^*_{\vec{k}} = \lambda_{-\vec{k}}. \tag{3.3}$$

The above formal expressions apply to the whole infinite space. In practice, in simulations we always deal with a finite region of a universe, sampled with a finite resolution. This means that we should use discrete versions of the above equations. We presented the continuous equations here, because they are often more convenient to use in various computations. It is, therefore, easier to do all computations in the continuous limit, and only switch to the discrete representation in the final step.

To simplify the presentation, we consider a somewhat limited but common case of a cubic volume discretized in each direction with N cubic cells (N^3 cells in total). If the grid spacing is Δx and the volume stretches from 0 to $L_{\rm box} = N\Delta x$ in each dimension, then, the spatial coordinates of grid centers are

$$\vec{x} = \Delta x\left[\left(i+\frac{1}{2}\right)\vec{e}_x + \left(j+\frac{1}{2}\right)\vec{e}_y + \left(k+\frac{1}{2}\right)\vec{e}_z\right],$$

[12] Unfortunately, the most widely used notation is somewhat ambiguous, with the same letter δ used for the density field and the Dirac delta-function. We will follow the standard practice, rather than to introduce a better but unfamiliar notation.

[13] To complicate the terminology even further, amplitudes $\lambda_{\vec{k}}$ are sometime called "phases", from a misuse of historical terminology and notation, where complex numbers $\lambda_{\vec{k}}$ were represented as absolute values and phases. Since, mathematically, they are amplitudes, we try to be as accurate as possible in our terminology.

where $\vec{e}_{x,y,z}$ are the unit vectors along the coordinate axes, and indices i, j, k take values from 0 to $N-1$, respectively. In order to compress notation, hereafter we will use the following short-hand definition,

$$\vec{e}_{ijk} \equiv i\vec{e}_x + j\vec{e}_y + k\vec{e}_z,$$

so that

$$\vec{x}_{ijk} = \Delta x \left[\vec{e}_{ijk} + \vec{e}_{\frac{1}{2}\frac{1}{2}\frac{1}{2}}\right]$$

and we use subscripts ijk to emphasize that $\vec{x}_{ijk}$ is set on a uniform grid.

The respective Fourier space (k-space) discretization is usually assigned at the vertices (rather than centers) of the uniform grid,

$$\vec{k}_{lmn} = \Delta k \vec{e}_{lmn},$$

where

$$\Delta k \equiv \frac{2\pi}{N\Delta x},$$

and indices l, m, and n go from $-N/2$ to $N/2-1$ (so that $k_{x,y,z}$ all go from $-k_{\rm Ny}$ to $k_{\rm Ny} - \Delta k$).

In a discrete form, equation (3.1) becomes

$$\delta(i,j,k) = \frac{1}{(N\Delta x)^{3/2}} \sum_{l,m,n=-N/2}^{N/2-1} \sqrt{P(k_{lmn})}\lambda_{lmn} e^{\mathsf{i}\vec{k}_{lmn}\vec{x}_{ijk}}, \tag{3.4}$$

and

$$\vec{k}_{lmn}\vec{x}_{ijk} = \frac{2\pi}{N}(il + jm + kn + (l+m+n)/2). \tag{3.5}$$

In equation (3.4) amplitudes λ_{lmn} are now normalized so that

$$\left\langle \lambda_{l_1 m_1 n_1} \lambda^*_{l_2 m_2 n_2} \right\rangle = \delta^K_{l_1 l_2} \delta^K_{m_1 m_2} \delta^K_{n_1 n_2}, \tag{3.6}$$

where δ^K_{ij} is the Kronecker symbol, i.e. individual λ_{lmn} are complex random numbers with a unit dispersion. If the initial density field is assumed to be Gaussian, λ_{lmn} are Gaussian random numbers distributed with zero mean and unit dispersion.

Equation (3.5) includes an extra phase in the exponential factor $\pi(l+m+n)/(2N)$, which arises from the convention of assigning the density field at the cell centers in the real space, but sampling the Fourier space at grid vertices. This extra phase, however, can be included into the random amplitudes by simply relabeling

$$\lambda_{lmn} e^{\mathsf{i}\frac{\pi}{2N}(l+m+n)} \to \lambda_{lmn}.$$

We use the following convention for continuous and discrete Fourier transforms:

Forward (real space to Fourier space)

$$f_k = \int dx f(x) e^{-\mathsf{i}kx}$$

$$f_l = \sum_{j=1}^{N} f(j) e^{-\mathsf{i}\frac{2\pi jl}{N}}$$

Backward (Fourier space to real space)

$$f(x) = \frac{1}{2\pi}\int dk f_k e^{\mathsf{i}kx}$$

$$f(j) = \sum_{l=-N/2}^{N/2-1} f_l e^{\mathsf{i}\frac{2\pi jl}{N}}$$

We always label the Fourier image of a function $f(x)$ with the same letter, but using the subscripts to refer to its argument, like in f_k.
In order to maintain consistent notation, we use a symbol i for the imaginary unit, to distinguish it from index i that we find difficult to avoid using.
Notice, that in our convention, the discrete Fourier transform is *not* normalized: applying forward and then backward transform will multiply the original function by N. This convention is followed by most numerical implementations of the discrete Fourier transform, including FFTW. We recommend the latter as the method to use for the FFT transforms in your code.

This relabeling does not violate conditions (3.3) and (3.6) imposed on the amplitudes, and is, therefore, always permitted. In fact, as can be easily checked, a similar relabeling is always permitted when the density field in real space is specified not at the cell centers, but at any other point inside a grid cell (as long as this point is at the same location relative to the cell center for all cells in the grid).

With this re-labeling, equation (3.4) becomes

$$\delta(i,j,k) = \frac{1}{(N\Delta x)^{3/2}} \sum_{l,m,n=-N/2}^{N/2-1} \sqrt{P(k_{lmn})}\lambda_{lmn} e^{\mathsf{i}\frac{2\pi}{N}(il+jm+kn)}. \tag{3.7}$$

At first it may appear that the number of "degrees of freedom" in the real and Fourier spaces in equation (3.7) are different. Indeed, since $\delta(i,j,k)$ is real, the delta field includes N^3 different real numbers, while in the Fourier space, the field $\sqrt{P(k_{lmn})}\lambda_{lmn}$ includes N^3 different *complex* numbers, or $2N^3$ different real numbers. In reality, everything is in order, since the Hermitian condition (3.3) imposes N^3 additional constraints in the Fourier space, so on both sides of the equation (3.7) there are only N^3 different real numbers.

Since most of existing numerical random generators produce real numbers, a complex random number with unit dispersion λ_{lmn} can always be constructed from two real random numbers with unit dispersion α_{lmn} and β_{lmn} as

$$\lambda_{lmn} = \frac{\alpha_{lmn} + \mathrm{i}\beta_{lmn}}{\sqrt{2}}.$$

An alternative way to specify the amplitudes λ_{lmn} has been proposed by Pen (1997), who pointed out that random amplitudes can also be specified in real space. Indeed, if $\lambda(\vec{x})$ are *real* random numbers that satisfy the independence condition,

$$\langle \lambda(\vec{x}_1)\lambda(\vec{x}_2)\rangle = \delta^3_D(\vec{x}_1 - \vec{x}_2),$$

then their Fourier transform,

$$\lambda_{\vec{k}} \equiv \int d^3x \lambda(\vec{x}) e^{-\mathrm{i}\vec{k}\vec{x}},$$

automatically satisfies the independence condition in Fourier space (3.2) *and* simultaneously satisfies the Hermitian condition (3.3).

At first, this may seem extraneous: why would one assign amplitudes in real space, and then Fourier transform then into a k-space, when they can be assigned in the k-space in the first place? The main reason is that assigning amplitudes in real space is much more flexible. This may not be that important if initial conditions are only assigned on a single, uniform grid, but if a series of simulation volumes is planned, of if some regions in the simulation volume need to be resolved better that the rest of the volume, or if the constrained initial conditions are needed, then assigning amplitudes in the real space becomes the method of choice[14]. We discuss constrained initial conditions and other applications of Pen's method below in §3.19 in detail.

Algorithmically, the whole process of computing $\delta(i,j,k)$ can be described as follows. First, as a universal common procedure, we generate density amplitudes in Fourier space at a moment a:

`procedure` **GenerateFourierAmplitudes**`[In:`a`;Out:`δ_{lmn}`]`

1. *For all (i,j,k), assign amplitudes in real space*

$$\lambda(i,j,k) = N(0,1), \tag{3.8}$$

   where call $N(0,1)$ generates a Gaussian random number with zero mean and unit dispersion.

2. *Fourier transform them*

$$\lambda_{lmn} = \texttt{Forward FT}[\lambda(i,j,k)].$$

3. *Scale amplitudes with the power spectrum*

$$\delta_{lmn} = \sqrt{P(k_{lmn}, a)}\lambda_{lmn}.$$

The realization of the density field can then be easily obtained by following a call to **GenerateFourierAmplitudes** with an inverse Fourier Transform,

`procedure` **GenerateLinearDensity**`[In:`a`;Out:`$\delta(i,j,k)$`]`

1. *Call* **GenerateFourierAmplitudes**`[In:`a`;Out:`δ_{lmn}`]`.

2. *Fourier transform into real space*

$$\delta(i,j,k) = \texttt{Backward FFT}[\delta_{lmn}].$$

[14]We generically call this Pen's method hereafter, although Pen Pen (1997) has not discussed all the application of this approach that we describe. However, the generalization of this approach to more complex situations is trivial.

Most of the FFT codes make the transform "in place" (i.e. without requiring substantial extra storage), and thus all four fields in the above example: $\lambda(i,j,k)$, λ_{lmn}, δ_{lmn}, and $\delta(i,j,k)$ can be located at the same memory location (i.e. to be the same array).

We also need initial velocity field, which can be used to set initial velocity of particles. Computing velocities is a straightforward generalization of the above procedure, because in the linear regime,

$$\vec{v}_{\vec{k}} = -\mathrm{i}a\frac{\dot{D}_+}{D_+}\frac{\vec{k}}{k^2}\delta_{\vec{k}}.$$

Notice, that in the above equation one divides by zero for the $k = 0$ mode. This mode can always be ignored, because it corresponds to a simple change of reference frame.

Algorithmically, generation of velocity field can be expressed as follows:

`procedure` **GenerateLinearVelocity**`[In:`a`;Out:`$\vec{v}(i,j,k)$`]`

1. *Call* **GenerateFourierAmplitudes**`[In:`a`;Out:`δ_{lmn}`]`.
2. *Compute Fourier transform of the velocity field*

$$\vec{v}_{lmn} = -\mathrm{i}a\frac{\dot{D}_+}{D_+}\frac{\vec{k}_{lmn}}{k^2_{lmn}}\delta_{lmn}.$$

3. *Fourier transform into real space*

$$\vec{v}(i,j,k) = \texttt{Backward FFT}[\vec{v}_{lmn}].$$

3.16.2 From the linear density field to the initial conditions

In practice, the initial conditions must be specified at a finite value of the cosmic scale factor $a_{\rm ini}$. This value is usually chosen so that the linear theory is already inaccurate at $a = a_{\rm ini}$, so the linear density and velocity field described in the previous section need to be modified appropriately.

In the simplest approach Zel'dovich approximation (see previous Chapter) is used to advance the linear theory fields to the initial scale factor $a_{\rm ini}$ (Doroshkevich et al., 1980; Klypin & Shandarin, 1983; Efstathiou et al., 1985). For the case of cold dark matter and Zel'dovich approximation, initial particle velocities are due solely to gravity and depend only on particle positions. In this case only 3D space needs to be discretized. For other type of dark matter (hot, warm, etc.), the initial velocity distribution must also be sampled as well.

Zel'dovich approximation uses a simple fact that in the non-relativistic linear regime the peculiar velocity $\vec{v}(\vec{x},a)$ grows with time as $a\dot{D}_+$. We can thus define a *displacement vector*

$$\vec{s} \equiv \frac{\vec{v}}{a\dot{D}_+}$$

which remains constant in the linear regime. Zel'dovich approximation assumes that in the non-linear regime the displacement vector remains constant in the *Lagrangian coordinates*, i.e. coordinates that remain comoving with the flow at all times. It is customarily to identify the Lagrangian position $\vec{q}$ for any particle (or streamline) with its initial spatial position,

$$\vec{q} \equiv \vec{x}(a=0).$$

The motion for a particle can then be described as

$$\begin{aligned} \vec{x}(a) &= \vec{q} + D_+\vec{s}(\vec{q}); \\ \vec{v}(a) &= a\dot{D}_+\vec{s}(\vec{q}). \end{aligned} \tag{3.9}$$

Depending on the time integrator method used, particle positions and velocities need to be specified at the same or different moments. For example, in a commonly used leap-frog integrator, particle positions and velocities are interleaved in the time variable (which does not have to be a physical time t, but can be, for

example, supercomoving time τ, scale factor a, log of the scale factor $\ln(a)$, etc). For generality, therefore, we consider setting particle positions and velocities at two values for scale factor, $a_{x,\rm ini}$ and $a_{v,\rm ini}$, which may or may not be the same.

Thus, algorithmically, the initial positions and velocities of all dark matter particles are specified with the following procedure.

`procedure` **GenerateDisplacementIC**`[In:`$a_{x,\rm ini}$`,`$a_{v,\rm ini}$`;Out:`$\vec{x}_l$`;`$\vec{v}_l$`;`m_l`]`

1. *Call* **GenerateFourierAmplitudes**`[In:`$a_{x,\rm ini}$`;Out:`δ_{lmn}`]`.
2. *Compute Fourier transform of the displacement vector field*

$$\vec{s}_{lmn} = -\mathsf{i}\frac{1}{D_+}\frac{\vec{k}_{lmn}}{k^2_{lmn}}\delta_{lmn}.$$

3. *Fourier transform into real space*

$$\vec{s}(i,j,k) = \texttt{Backward FFT}[\vec{s}_{lmn}].$$

4. *Sample the displacement field from the uniform grid $\vec{s}(i,j,k)$ onto initial particle positions,*

$$\vec{s}(i,j,k) \to \vec{s}_l.$$

5. *Move particles from their initial positions $\vec{q}_l$ and set their velocities using Zel'dovich approximation,*

$$\begin{aligned}\vec{x}_l &= \vec{q}_l + D_+(a_{x,\rm ini})\vec{s}_l;\\ \vec{v}_l &= a_{v,\rm ini}\dot{D}_+(a_{v,\rm ini})\vec{s}_l.\end{aligned}$$

6. *Assign all particles identical masses (if needed),*

$$m_l = m_0.$$

Note that in the algorithm above we use the same displacement vector, computed from the power spectrum at $a = a_{x,\rm ini}$. This is only valid if the transfer function T_k does not depend on the scale factor at the initial moment (which is the case for the vast majority of cosmological models). For exotic physical models, where the transfer functions strongly evolve at the initial moment of the simulation, the displacement vectors need to be computed at two separate moments $a_{x,\rm ini}$ and $a_{v,\rm ini}$.

Besides Zel'dovich approximation, which is, formally, a linear perturbation theory in the Lagrangian space, higher order approximations can also be used Crocce et al. (2006).

The last minor point that needs attention here is step 3 in procedure **GenerateDisplacementIC**. For the grid initial placement, if the number of particles is equal to the number of cells on the grid, no interpolation is required. For the glass placement (see next section), displacements need to be interpolated from the grid to the particle locations. Depending on the way the interpolation is done, a small amount of small-scale smoothing can be introduced by the interpolation method.

3.17 Pre-initial particle placement.

Given that the density distribution in an N-body simulation is sampled by a finite number of particles, their initial placement should be optimized to faithfully reproduce properties of the desired density field over the widest possible range of scales and to minimize the discreteness effects associated with a finite particle number. A related problem of "quiet start," ensuring correct growth rate of instabilities, is well-known in plasma simulations Byers & Grewal (1970) and N-body modeling of disk systems Sellwood (1983).

Here we discuss different approaches to choosing initial Lagrangian coordinates of particles, $\{\mathbf{q}_i\}$ (or "pre-initial" positions). The criterion here is minimizing the power of large-scale fluctuations of the resulting density fields: namely, the $P(k)$ of distribution of particles located at $\{\mathbf{q}_i\}$. For example, the power spectrum

of a random, uniform distribution of particles[15] is flat (white noise), $P(k) = \text{const} = \text{V}_{\text{box}}/\text{N}_{\text{p}}$, where $V_{\text{box}} = L^3_{\text{box}}$ is the comoving volume of the simulation box and N_{p} is the total number of particles. Such distribution is of course exactly what is needed for modelling of a $P(k) \propto k^0$ density field and in fact has been used to study growth of large-scale structures in some of the early cosmological simulations Miller (1983, 1984). For density fields with other power spectra, however, placing particles randomly is not optimal and is generally a bad idea, because the white noise power can be larger than the target power, $P(k)$ for all the modes, k for which $P(k) < V_{\text{box}}/N_{\text{p}}$. The initial power of the white noise is real and gravity will lead to its amplification when the distribution is evolved Miller (1983, 1984). The uniform distribution of particles in an overdense sphere, for example, was typically used as initial conditions for simulations of "cold collapse" Peebles (1970). Such setup leads to a visible growth of the small scale structures corresponding to the small-scale power of the white noise in the initial conditions. Poisson pre-initial placement of particles should therefore be avoided in general in the cases when the initial power spectrum is not supposed to be flat.

A choice representing an opposite extreme to the uniform random distribution is to place particles on a regular lattice with spacing $L_{\text{box}}/N_{\text{p}}^{1/3}$:

$$q_{x,i} = (i-1)\frac{L_{\text{box}}}{N_{\text{p}}^{1/3}}, \quad \text{for} i = 1, ..., N_{\text{p}}^{1/3},$$

with similar placing of y and z coordinates[16] This is actually the most commonly used pre-initial particle placement (e.g., Klypin & Shandarin (1983); Efstathiou et al. (1985)). Such a distribution has $P(k) = 0$ for $k < k_{\text{Ny}}$. The price is a strong anisotropy of the particle distribution at the scales close to interparticle separation. This not only goes counter to the assumption of isotropy of the Universe encoded in the Cosmological Principle, but also results in a series of spikes in $P(k) = V_{\text{box}}$ at wavenumbers $k_s = n2k_{\text{Ny}}$ ($k \neq 0$, and $n = 1, 2, ...$). If the realization of the density field is generated by displacing particles from the original lattice, the spectrum will change. In general, displacing particles from a regular lattice to reflect a given input power spectrum, $P_{\text{inp}}(k)$, will result in a density field with the power spectrum equal to the sum of $P_{\text{inp}}(k)$ and the power spectrum arising from the discreteness of mass distribution (Joyce & Marcos, 2007).

Note that when particles are shuffled with respect to the initial grid positions at $k > k_{\text{Ny}}$ the the average power spectrum will approach that of the white noise $P(k) = V_{\text{box}}/N_{\text{p}}$ for increasingly large k, regardless of the pre-initial particle placement. For example, if we split simulation volume into cells of size much smaller than the interparticle separation, the occupation number for each cell will be either 1 or 0 and the associated density PDF will be described by the Poisson distribution. This is an irreducible effect of discreteness. However, this white noise power *does not grow under the action of gravity* if these scales are not resolved by N-body code. Overall, pre-initial placement of particles on a regular grid works well and does not result in quantitatively incorrect results for most applications. However, in this case one can often still see the residual regular pattern imprinted by such placements in the low-density void regions of evolved distributions. Thus, for aesthetic reasons some authors have proposed different algorithms for pre-initial placement which do not have regular pattern and minimize power on the largest scales.

Zel'dovich (1965) pointed out that the minimal fluctuations due to discreteness in matter distribution are characterized by the spectrum $P(k) \propto k^4$ (see also § 28.A and 28.B in Peebles (1980) for a more detailed discussion). The algorithms for the initial placement of particles should therefore aim to generate particle distributions with the power spectra as close as possible to $P(k) \propto k^4$.

An algorithm that achieves this goal very closely uses random distribution of particles evolved to a "glass"-like distribution with an N-body code using using negative accelerations (repulsive gravity White, 1996). For example, Smith et al. Smith et al. (2003) show that pre-initial glass-like particle distribution is characterized by a power spectrum $P(k) \propto k^\beta$ with $\beta \approx 3.8$ at $k \lesssim k_{\text{Ny}}$ (i.e. quite close to the optimal), while at $k > k_{\text{Ny}}$ it transitions to the white noise spectrum: $P(k) = \text{const}$. Note, however, that the resulting power spectrum does depend on the details of how glass distribution is generated. For example, Baugh et al. Baugh et al. (1995) find power spectrum considerably flatter than k^4 at $k < k_{\text{Ny}}$ in their realization of the glass initial conditions (see their Figure A2).

A different "quaquaversal" placement has been proposed by Hansen et al. (2007). This placement achieves the theoretical $P(k) \propto k^{-4}$ power spectrum, but was shown to lead to more significant numerical fragmentation problems in the warm dark matter (WDM) simulations compared to glass and grid placement (Wang & White, 2007).

[15] Generated by assigning each particle coordinate a uniform random variable in the range $[0, L_{\text{box}}]$.

[16] One can choose slightly different placements, if particles are chosen to be placed in the centers of grid cells, for example: $q_{x,i} = (i - 0.5)L_{\text{box}}N_{\text{p}}^{1/3}$.

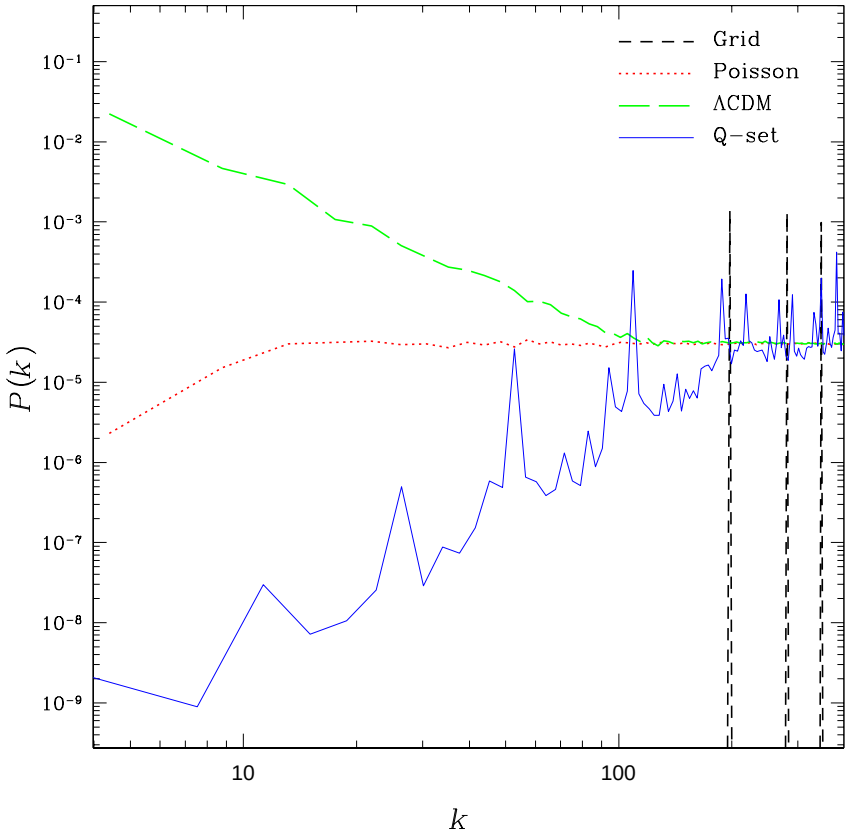


Figure 3.3: The power spectra for a quaquaversal pre-initial placement (solid blue line), uniform grid placement (short-dashed black line), a Poisson dis- tribution (dotted red line) and a CDM initial con- dition (long-dashed green line). Adopted from Hansen et al. (2007).

Why and where the initial placement method matters? In CDM models one can wait until real power overtakes the spurious discreteness power because discreteness power does not evolve Smith et al. (2003). In addition, the grid and glass pre-initial placements are producing statistically indistinguishable power spectra Baugh et al. (1995); Smith et al. (2003), at least at the scales at which the real power spectrum dominates over the white noise. Nevertheless, there are striking visual differences in the particle distributions evolved from the grid and glass initial conditions, especially in the low-density regions where the initial grid-like pattern persists to the late stages of evolution. Thus, although the differences do not manifest strongly in two-point statistics, such as the power spectrum, the effects on other statistics or properties of collapsed structures have not yet been explored. Glass like pre-initial distribution with a power spectrum as close to $P(k) \propto k^4$ at $k < k_{\rm Ny}$ as possible is therefore preferred in general.

This is particularly true for the simulations of the Warm Dark Matter models, in which the real power spectrum is truncated below some scale $k_{\rm trunc}$ and one would like to follow the evolution of structure resolving this scale (i.e., $k_{\rm Ny} > k_{\rm trunc}$). Götz & Sommer-Larsen (2003); Wang & White (2007). However, Wang & White (2007) showed that none of the placement methods completely precludes artificial fragmentation in filamentary regions due to particle discreteness noise. Hahn et al. (2013) show that numerical fragmentation can be almost completely eliminated by improving density estimate method from particle distribution during N-body evolution. Another class of applications, in which initial placement may be important is cold collapse of initial perturbations. In this case, one would want to start with "quiet" initial conditions which minimize collapse of small-scale sturcture due to spurious Poisson fluctuations within the initial perturbation.

As an exercise, it is useful to test whether pre-initial particle placement matters in your simulations. To do this, you can use the PM code you have developed with the sign of accelerations inverted (i.e. repulsive instead of attractive gravity). To this end, generate Poisson distribution of particles in the target box with

zero velocities. The number of grid cells in the box should be at least eight times larger than the particle number, so that nearby particles feel mutual forces. Run the PM code with accelerations of opposite sign and measure power spectrum of intermediate particle distributions. The power spectrum should be flat in the start of simulations, but should steepen with time. Stop the simulation when power spectrum does not exhibit any appreciable further evolution.

3.18 The effects of finite box size

When setting up cosmological simulations and analyzing and interpreting their outputs one should always keep in mind numerical limitations imposed by discreteness and a finite size of simulation box. Here we discuss particular effects and biases that one needs to be aware of.

The finite size of the simulation volume is a numerical limitation of a simulation, and, as such, results in numerical artifacts and incorrect representation of the matter distribution on some scales. The following two effects are especially important: (1) the discreteness and the limited range of the k-space sampling and (2) the sample variance of the large-scale fluctuation modes, which manifests itself in the deviations of the power spectrum for a particular realization of the density field on the largest scales from the true power spectrum.

3.18.1 Discreteness of k-space

Uniform sampling in k-space becomes highly uneven on the largest scales in real space: the largest value of k sampled is the fundamental mode $k_1 = \Delta k N$, which is realized with just 3 vectors in the Fourier space: $\vec{k} = k_1(1,0,0)$, $\vec{k} = k_1(0,1,0)$, and $\vec{k} = k_1(1,0,0)$). The next value of the wavenumber k realized on a uniform grid is $k_2 = k_1\sqrt{2}$ (realized by vectors similar to $\vec{k} = k_1(1,1,0)$). These two nearest sampled values correspond to spatial scales of $l_1 = 2\pi/k_1$ and $l_2 = 2\pi/k_2$ which are $\sqrt{2}$ apart. Values of the wavenumber k between k_1 and k_2 are not sampled on the grid, and if the power spectrum happen to have a feature inside that interval, it will not be reflected in the generated initial conditions.

This discreteness also leads to another, even more serious artifact: the power for $k < k_1$ is not sampled at all, i.e. all power on spatial scales above l_1 is not included in the simulation - this is knows as a "missing large scale power problem", and is a severe limitation in simulations with small box sizes.

The root of this problem is the condition $P(0) = 0$ for the true linear power spectrum. That conditions is appropriate for the infinite universe, where the mean overdensity $\langle\delta\rangle$ should be zero. However, for a finite box this is not the case - for any finite box size L the average density in the simulation box is not in general equal to the mean density of the universe, but is related to the amount of power on scale L. In other word, the mean overdensity in a finite box is *not*, in general, zero,

$$\langle\delta\rangle_L \neq 0.$$

This non-zero value for the mean overdensity is sometimes called a "DC mode", in analogy to the AC/DC electric current, since it is just a constant.

The flip-side of this issue, is the fact that the correlation function over the entire Universe must be zero, which means that there should be at least one zero-crossing scale, r_0, where correlation function changes sign. For a concordance cosmology, this scale is about $r_0 \approx 120h^{-1}\,\mathrm{Mpc}$. It is clear that for boxes comparable or smaller to this scale, zero crossing will be artificially forced to smaller scales. As the result, on large scales various real-space statistical measures, such as variance in spheres or correlation functions, are not represented properly. This is of serious concern because it is the real space density fluctuations that drive the growth of structures. In fact, empirically, correlation function in the simulations initialized in this way misrepresent the true correlation function at scales $r \gtrsim 0.1 L_{\mathrm{box}}$ (Colin et al., 1997). If the affected scales become nonlinear, the errors will propagate to smaller scales due to mode coupling and will affect the real-space correlations to even smaller scales.

In order to circumvent this deficiency and properly include the DC mode in the initial conditions, the input power spectrum has to be modified to permit a non-zero value at $k = 0$. As a solution, Pen (1997) recommends using the "grid" power spectrum instead of the true one - i.e., since the grid introduces artifacts anyway, it may be more beneficial to hide these artifacts in k-space, while maintaining the accuracy of real-space statistics all the way to the box size.

The grid power spectrum is a convolution of the true power spectrum $P(k)$ with a window function that reflects the discreteness of the simulation box,

$$P_{\rm grid}(\vec{k}) \equiv \int d^3k' P(k') W_C(\vec{k} - \vec{k}'), \tag{3.10}$$

where W_C is the window function of a cube of size L,

$$W_C(\vec{k}) = \frac{\sin(k_x L/2)\sin(k_y L/2)\sin(k_z L/2)}{\pi^3 k_x k_y k_z}.$$

The grid power spectrum is not isotropic, it depends on the full wavevector $\vec{k}$ rather than on its amplitude k only. Pen (1997) also noted that using $P_{\rm grid}(\vec{k})$ instead of the "true theoretical" $P(k)$ improves several other statistical measurements on scales close to the box size.

The only - but very serious - limitation in using $P_{\rm grid}(\vec{k})$ is that it requires computing a 3-dimensional integral over Fourier space. The integral (3.10) converges very slowly (and Cartesian coordinates are better than spherical coordinates), so that computing the grid power spectrum for a sufficiently large grid, even taking into account the symmetries of W_C, becomes a highly non-trivial numerical undertaking, potentially rivaling the computational costs of the simulation itself.

A practical way of computing $P_{\rm grid}$ has been suggested by Pen (private communication). Instead of computing the 3D convolution integral, one can compute the linear correlation function $\xi(r)$ on the grid in real space, and then Fourier transform it into k-space to obtain a highly accurate approximation for $P_{\rm grid}$. There exist several possible methods for setting $\xi(r)$ on a regular grid. We recommend setting $\xi(r)$ at the vertices of the grid, measuring r from the box corner, and taking into account periodic boundary conditions,

$$r(i,j,k) = \left[\min(i, N-i)^2 + \min(j, N-j)^2 + \min(k, N-k)^2\right]^{1/2},$$

where indicies i, j, and k run from 0 to $N-1$, as before. This approach requires computing $\xi(0)$, which diverges for many linear power spectra. Because the grid has a finite spatial spacing Δx, the smallest scales are not captured on the grid, and $\xi(0)$ needs to be computed by Fourier transforming a square root of the input power spectrum,

$$\xi(0) \equiv \left[\frac{1}{(N\Delta x)^{3/2}} \sum_{l,m,n=-N/2}^{N/2-1} \sqrt{P(k_{lmn})}\, e^{\,\mathrm{i}\frac{2\pi}{N}(il+jm+kn)}\right]^2.$$

Alternatively, one can sample the correlation function at cell centers,

$$r(i,j,k) = \left[\min(i+1/2, N-1/2-i)^2 + \min(j+1/2, N-1/2-j)^2 + \min(k+1/2, N-1/2-k)^2\right]^{1/2},$$

but since the smallest scale sampled in this case is $\sqrt{3}\Delta x/2$, this introduces small scale smoothing in the initial conditions.

A simplified approach was proposed by Sirko (2005), who pointed out that if requirement of the correct $\xi(r)$ is restricted to a sphere with the radius $L/2$ (half the box size), then the grid power spectrum becomes a simple 1D convolution of the input linear power spectrum $P(k)$. We will call such a restricted convolution a "ball" power spectrum,

$$P_{\rm ball}(k) \equiv \int_0^\infty P(k') W_{\rm Ball}(k', k) k'^2 dk', \tag{3.11}$$

where

$$W_{\rm Ball}(k', k) = \frac{2}{\pi} \frac{k'\cos(k'L/2)\sin(kL/2) - k\cos(kL/2)\sin(k'L/2)}{kk'\left(k^2 - k'^2\right)}.$$

This convolution is easy to compute numerically with any standard numerical integrator. The price one pays for the numerical simplicity is that $P_{\rm ball}(k)$ only gives accurate values for the mass variance in spheres of $R < L/4$, while the grid power spectrum maintains accurate mass variance up to $R < L/2$. In practice this method is not faster than using the grid power spectrum, computed by Fourier transforming the linear correlation function, so we see no reason to recommend it.

Comparison between these three choices for the input power spectrum is presented in Figure 3.4, where we show σ_8 as a function of the simulation box size for a ΛCDM cosmological model. As expected, using exact

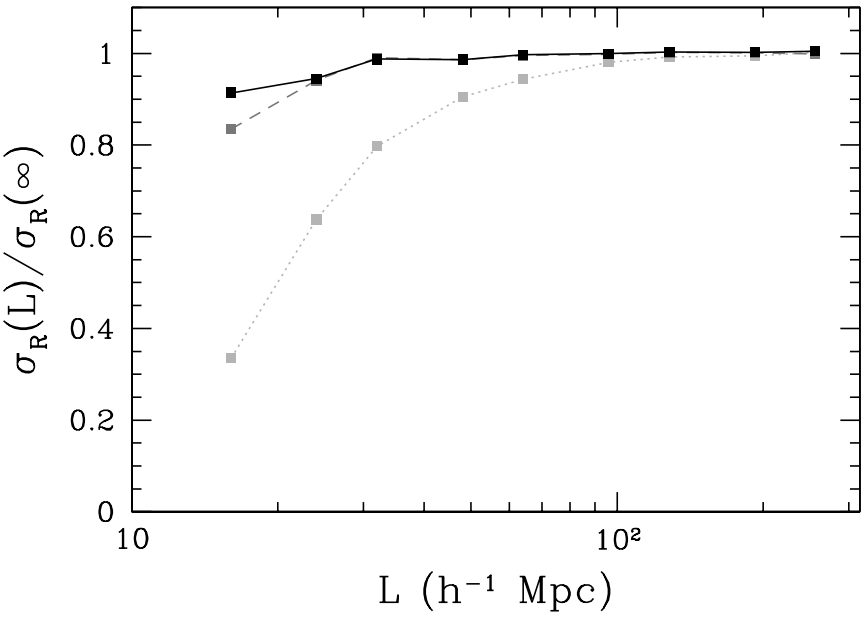


Figure 3.4: The mass variance in $8h^{-1}$ Mpc spheres σ_8 as a function of the simulation box size L, as measured from a large number of realizations. Light gray dotted line shows the standard method, with the true linear $P(k)$ used in Step 3 of procedure **GenerateFourierAmplitudes**. Dark gray dashed line is for the power spectrum corrected in spheres, $P_{\rm ball}(k)$ (equation 3.11), while the solid black line is for the grid power spectrum $P_{\rm grid}(k)$ (equation 3.10).

linear power spectrum results in substantial losses of large scale power for box sizes as large as $100h^{-1}$ Mpc ($R \lesssim L/15$). Both, grid and ball power spectra begin to lose large scale power slightly below $32h^{-1}$ Mpc ($R < L/4$) due to the discrete sampling of k-space on the grid; using the ball power spectrum leads to further power losses at $16h^{-1}$ Mpc ($R < L/2$), as expected from its definition.

We, therefore, recommend using $P_{\rm grid}$ for simulations with small boxes (i.e. with boxes where fluctuations on the box scale at the final moment of the simulations are comparable to the precision required from a simulation), or for simulations for which σ_8 needs to be highly precise, like simulations of galaxy clusters (if the box size is less than about $150h^{-1}$ Mpc).

3.18.2 Large-scale power distribution

Another artifact - or, rather, "a feature" - is due to the fact that large-scale waves in a finite box are sampled by a small set of amplitudes. For example, as we mentioned in § 3.18.1, the fundamental mode k_1 is represented by only three independent waves: $\vec{k} = k_1(1,0,0)$, $\vec{k} = k_1(0,1,0)$, and $\vec{k} = k_1(1,0,0)$. If the amplitude of each waves is Gaussian distributed, the power at the fundamental mode is distributed as a χ^2 distribution with 3 degrees of freedom, and, therefore, fluctuates substantially.

In particular, this means that if you setup your initial conditions with some power spectrum $P(k)$, and then *measure* the power spectrum from the linear density field, you will *not* get $P(k)$ back, but rather a function that will fluctuate around $P(k)$, with the amplitude of fluctuations decreasing toward smaller scales. This is illustrated in Figure 3.5, where we show the input power spectrum for a ΛCDM model, as well two power spectra as measured from two density realizations on a grid: the first realization is truly random, and the second one is selected among 1000 random ones to minimize the maximum deviation between the measured and the input power spectra.

The bottom panel of Fig. 3.5 shows the mean and the rms deviation of a measured power spectrum from the exact input one, as computed from 1000 random realizations. The measured power spectrum depends somewhat on how one actually measures it - we show two measurements, with k-space sampled uniformly with $\Delta k = k_1/2$ and $\Delta k = k_1$. In the first case, only the fundamental mode contributes to the lowest bin $0.75k_1 < k < 1.25k_1$, while in the second case the next discrete mode $k_2 = k_1\sqrt{2}$ (produced by k vectors like $\vec{k} = k_1(1,1,0)$) also contributes to the first bin $0.5k_1 < k < 1.5k_1$.

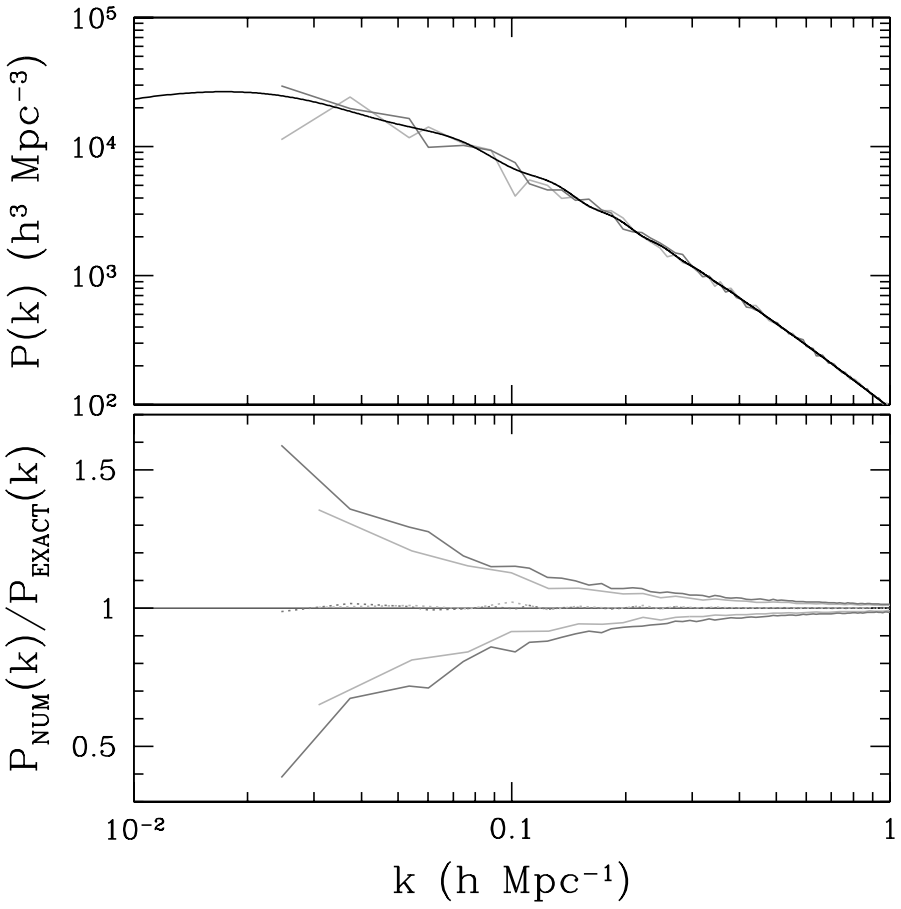


Figure 3.5: Top panel: power spectrum at $z = 0$ for a ΛCDM model (black), together with power spectra from two random realizations (256^3 grid with $1h^{-1}$ Mpc cell size). The light gray spectrum is typical, while the dark gray one is chosen from 1000 random realizations to minimize the maximum deviation from the true input spectrum. Bottom panel: the mean (dotted) and rms fluctuations (solid) in the ratio of the power spectrum realized on a 256^3 grid with $1h^{-1}$ Mpc cell size to the true input power spectrum. The average and the rms fluctuation are computed from 1000 random realizations. Light and dark gray sets of lines show different sampling of k-space: dark gray lines are for sampling in bins of $\Delta k = k_1/2$, while light gray lines are for twice coarser, $\Delta k = k_1$ sampling.

Irrespectively of how one actually measures the power spectrum from a specific realization of the linear density field, rms fluctuations of the order of 50% in the power at the fundamental mode should be expected - and this conclusion does not depend on whether one uses a pure linear $P(k)$ or box convolutions like $P_{\rm grid}$ (3.10) or $P_{\rm ball}$ (3.11).

3.19 Constrained initial conditions

In some cases, a simulated region of the universe is taken to be not entirely random (as assumed in §3.16.1). In that case the initial conditions set in such a region are said to be "constrained". For example, the constrain may be that a sufficiently large dark matter halo is located at the center of the box, or that the simulation volume resamples with higher resolution a region from another simulation.

Historically, several methods and their modifications were introduced to achieve such constraints (Bertschinger, 1987; Hoffman & Ribak, 1991; Salmon, 1996). A considerably more flexible method of setting random amplitudes in real space introduced by Pen (1997) permits to create arbitrarily constrained initial conditions at no extra computational cost.

The key step in using real space amplitudes in setting constrained initial conditions is the "refinement" of a lower resolution initial condition to a higher resolution one by adding small-scale power. Let us imagine

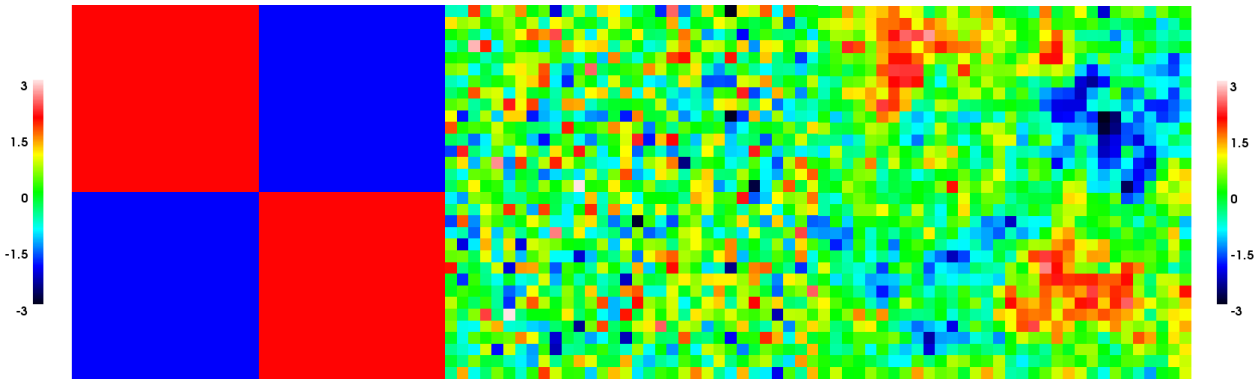


Figure 3.6: An example of constrained initial conditions using real space amplitudes. The left panel shows the constraint: $\lambda_1 = [2, -2, -2, 2]$. The middle panel are constrained random phases (3.13), and the right panel are the resultant density field $\delta(i, j, k)$ with $P_{2D}(k) \propto k^{-2}$ (equivalent to $P_{3D}(k) \propto k^{-3}$). The effect of the constraint is easily visible - the fluctuations have higher amplitudes on large scales along the diagonal. The color-bars show fluctuations in units of their rms fluctuations.

that we have a realization of the random amplitudes $\lambda_1(i, j, k)$ on a cubic grid of the size N_1 in each dimension (again, the case of a non-cubic rectangular grid is a trivial generalization). Our goal is to refine the amplitudes to a higher resolution one, with $N_2 = M \times N_1$ cells in each direction, where M is an integer number. In other words, each cell (i_1, j_1, k_1) in the low resolution realization is refined into M^3 cells in the high resolution one (with indices $i_2 = Mi_1, ..., Mi_1 + M - 1$, etc.).

Let new amplitudes be $\lambda_2(i, j, k)$. Then for each set of indices $i_2 = Mi_1, ..., Mi_1 + M - 1$, etc, in the lower resolution grid, first assign the Gaussian random numbers for all cells and then correct the amplitude

$$\begin{aligned} \hat{\lambda}_2(i_2, j_2, k_2) &= N(0, 1) \\ \lambda_2(i_2, j_2, k_2) &= \hat{\lambda}_2(i_2, j_2, k_2) - \frac{1}{M^3} \sum_{i,j,k=0}^{M-1} \lambda_2(Mi_1 + i, Mj_1 + j, Mk_1 + k) + \frac{1}{M^{3/2}} \lambda_1(i_1, j_1, k_1), \end{aligned} \tag{3.12}$$

where $N(0, 1)$ is again a Gaussian random number with zero mean and unit amplitude, as in equation (3.8). In this formalism, a constraint on the random amplitudes is simply a replacement of the mean amplitudes in all M^3 boxes with the values from the constraint field λ_1 (divided by $M^{3/2}$ - the latter factor comes from the normalization constraint $\langle \lambda_2^2 \rangle = 1$).

Thus, in the Pen's method, arbitrary constrained initial conditions can be generated with the same procedure we described above, with a simple change of equation (3.8) in step 1 of procedure **GenerateFourierAmplitudes** with equation (3.13). Easy, isn't it?

In order to illustrate the versatility of Pen's method, we show a few toy 2D examples of its different applications. Figure 3.6 illustrates the direct application of this method. The left manel shows the constraint: on the scales of 1/2 of the box size the density fluctuations are $+2\sigma$ (recall, σ is always 1 for amplitudes) along the diagonal, and -2σ off the diagonal. The middle panel shows the constrained random amplitudes λ_2 on a 32×32 grid ($N_1 = 2$, $M = 16$, so $N_2 = 32$). Because the random amplitudes are uncorrelated, no constraint is noticeable in their distribution. However, in the resultant density field (the right panel) the effect of the constraint is obvious.

The next example (Figure 3.7) is a more practical one - a lower resolution initial conditions are refined to higher resolution. In this case λ_1 are also uncorrelated random amplitudes, and equation (3.13) is used to add small-scale power that is missing in the lower resolution realization.

Figure 3.8 is another example of constrained initial conditions. The constraint field (left panel) does not average to zero, but in Pen's method it does not have to. The density realization has a zero mean, because we have not used the grid power spectrum $P_{\rm grid}$ in these examples, but rather set $P(k) = 0$ for simplicity.

Another useful example of Pen's method is shown in Neyrinck et al. (2004), where an hierarchy of nested simulation boxes modeling the same objects at their centers with different resolutions was constructed by successfully refining centers of lower resolution boxes.

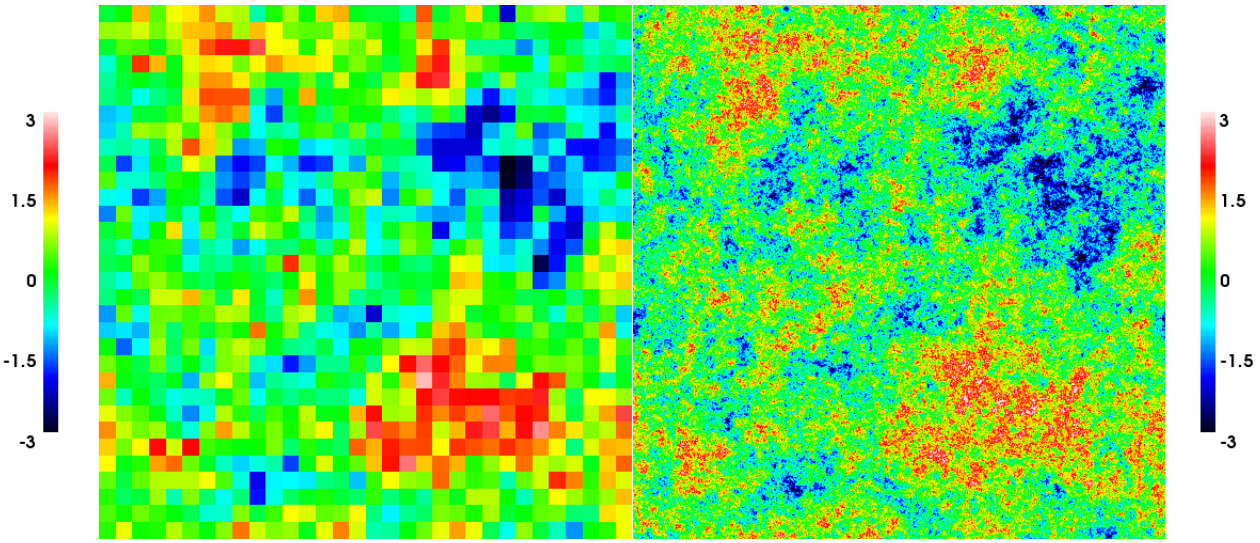


Figure 3.7: An example of using Pen's method to refine a lower resolution initial conditions. The left panel shows the low resolution ($N_1 = 32$) box, while the right panel is for the initial conditions refined by a factor $M = 16$ ($N_2 = 512$).

3.20 Selectively refined ("zoom-in") initial conditions

For some applications, such as simulations or simulations with multiple particle species with varied masses, a realization of the initial density field with different spatial resolution in different regions of the simulation volume may be needed. In principle, one can apply a brute force method to this problem and generate a uniform IC on a full grid and then sub-sample it in the regions where high-resolution is not needed (e.g., Klypin et al., 2001).

Pen's method discussed above can also be used to create the refined initial conditions with ease. The starting point for such a realization is the set of two uniform, mutually consistent realizations of the density field: a low resolution one and a high resolution one, as shown in Fig. 3.7 (let's call them δ_1 and δ_2). Using the mask from the left panel of Fig. 3.8 ($m(i,j,k)$), these two realizations can be combined as

$$\delta(i,j,k) = \begin{cases} \delta_2(i,j,k) & \text{if } m(i,j,k) == 1, \text{ and} \\ \delta_1(i/M, j/M, k/M) & \text{if } m(i,j,k) == 0. \end{cases}$$

Such a selectively-refined realization is shown on the left panel of Fig. 3.8. Note that you do need two realizations - simply taking the high resolution one and averaging it in the un-selected regions in blocks of M^3 cells each leads to lower amplitude fluctuations in the un-refined region - as can be seen on the right panel of Fig. 3.8. The reason for this is that averaging procedure actually modifies the power spectrum of fluctuations, resulting in lower power over a range of scales close to the Nyquist frequency.

The only limitation of this approach is that we need a uniform realization of initial conditions with the highest resolution of the refined realization. At present, it is possible to create single realization with resolution up to 1024^3 on a high-end workstation. But the computational cost of creating such a realization grows very fast with the resolution: a 8192^3 realization is 512 times more expensive - 512 high-end workstations are equivalent to a serious supercomputer. Therefore, there is a practical limit (depending on one's computing resources) to how high a resolution one can achieve in this approach.

If exceptionally high resolution in refined regions is needed, a different approach is required. If all refined regions can be set as cubes, then separate Fourier transforms inside them can be used. Such an approach is, however, far from straightforward, because periodic boundary conditions have to be handled with special care. Details of this approach are beyond the scope of this tutorial. There exists a publicly available code (*GRAFIC2*) that implements it (Bertschinger, 2011). The code description and the full mathematical formalism behind it are given in Bertschinger (2001).

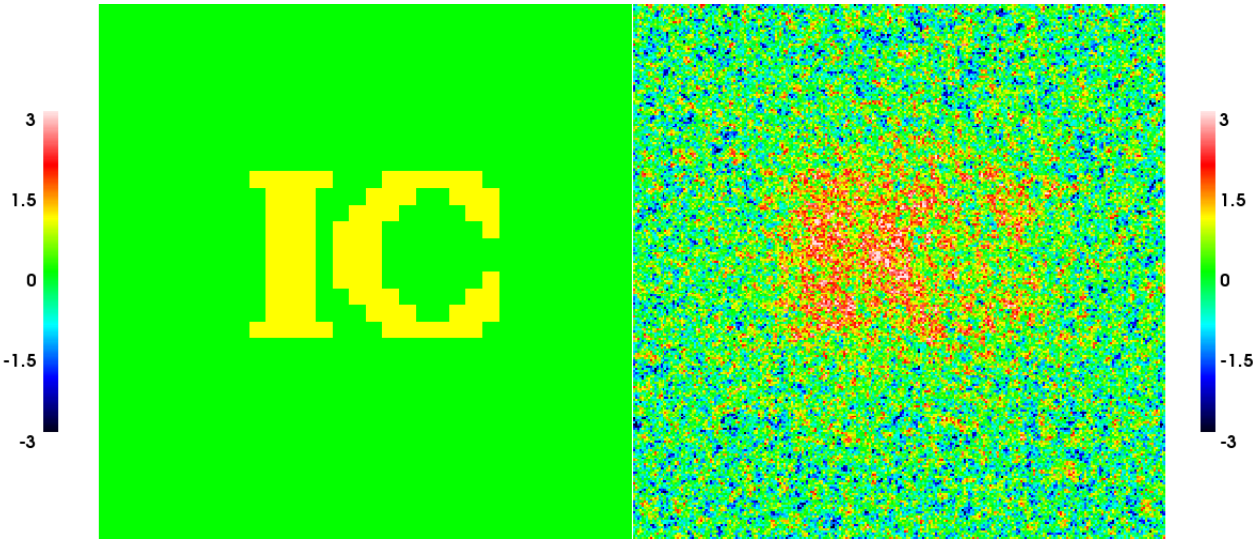


Figure 3.8: Another example of constrained initial conditions. The constraint field is on the left, and the density realization is on the right.

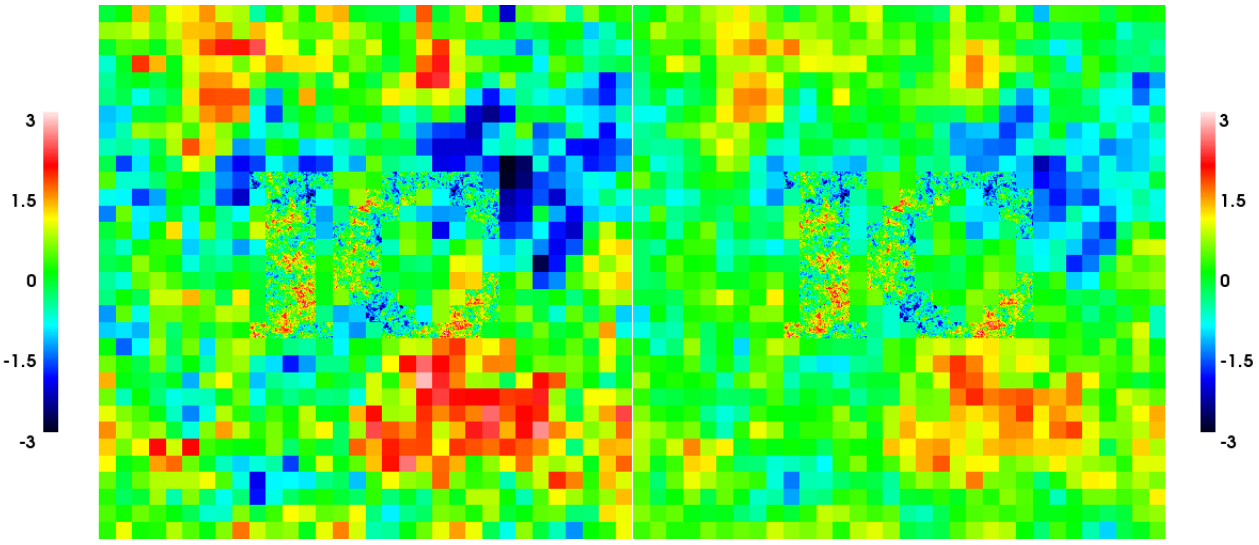


Figure 3.9: A selectively refined realization of initial conditions (left). The refinement mask is given by the left panel of Fig. 3.8. The right panel shows instead a uniform high resolution realization, averaged over M^3 cells in the un-selected region. Notice how averaging reduces the power in the unrefined region.

Power spectrum

In this Chapter we provide the main information about input power spectrum in standard cosmological models.

4.21 Linear Power Spectrum

4.21.1 Theory

For a Gaussian random field, the power spectrum of fluctuations fully determines the statistical properties of the initial conditions. In expanding universe, the power spectrum of density fluctuations at any moment t (or scalar factor a) can be separated into two factors that encapsulate two different physical processes:

1. the primordial power spectrum $P_p(k)$, generated in the early universe, and
2. a factor $G_p(k, a)$ that describes the evolution of fluctuations at scale k from the early universe to the moment a.

While the former factor depends on the early universe physics (and, therefore, may be poorly known), the latter one can, in principle, be computed by a standard cosmological Boltzmann code.

The form $P(k, a) = P_p(k)G_p(k, a)$ is physically self-evident, but absolutely useless practically, since it requires a precise specification of what the "early universe" actually means, and a computation of G_p throughout various early universe stages becomes a nightmare.

Instead, we can come up with an alternative form for $P(k, a)$, which uses some other, more recent moment as a reference instant in time. For example, one can use the present time. In practice, however, this is not convenient, since, due to the presence of the dark energy, density fluctuations may grow in a complex way around at the present time. Instead, it is customary to define the power spectrum in a slightly more convoluted way, but which is more convenient for numerical calculations.

Namely, we define

$$P_0(k) \equiv P_p(k)\frac{G_p(k \to 0, a_*)}{\mathcal{D}_+(a_*)^2}$$

as the primordial power spectrum, scaled up to some reference moment a_* by the amount of growth that occurs at large scaled (small k), and then further scaled by a factor $\mathcal{D}_+(a_*)^2$ where the function $\mathcal{D}_+(a)$ is *defined* as

$$\mathcal{D}_+(a) = a + \frac{2}{3}a_{\rm eq} + \frac{a_{\rm eq}}{2\ln(2) - 3}\left[2\sqrt{1+x} + \left(\frac{2}{3} + x\right)\ln\frac{\sqrt{1+x}-1}{\sqrt{1+x}+1}\right], \tag{4.13}$$

where $x \equiv a/a_{\rm eq}$ and $a_{\rm eq} \equiv \Omega_R/\Omega_m$ is the scale factor of matter-radiation equality. The last two terms can often be neglected, the third one falls below 1% for $z < 1{,}000$ and the second one is below 1% for $z < 30$.

The reason for such a complex definition is that in the universe that contains only matter and radiation, $G_p(k \to 0, a) \propto \mathcal{D}_+(a)^2$, and, thus, if a_* is chosen during the matter-dominated era, then $P_0(k)$ is actually independent of that choice. It is also independent of the details of the dark energy evolution at recent times.

In the literature, P_0 is still often called the "primordial power spectrum", although it relates to the reference time a_* and is *not*, in fact, a power spectrum of the density fluctuations at any moment in the history of the universe (since it does not include the full $G_p(k, a)$). We follow the customs of the field and call P_0 the "primordial power spectrum" from now on, although, strictly speaking, the term is a misnomer.

We can now represent $P(k, a)$ as

$$P(k, a) = P_0(k)\left[\mathcal{D}_+(a_*)^2\frac{G_p(k \to 0, a)}{G_p(k \to 0, a_*)}\right]\left[\frac{G_p(k, a)}{G_p(k \to 0, a)}\right]. \tag{4.14}$$

The two terms in square brackets are separated for historical reasons. The second term is usually represented by the *transfer function* T_k for the density fluctuations (which may be different for different components: dark matter, baryons, neutrinos, etc),

$$\left[\frac{G_p(k,a)}{G_p(k\to 0,a)}\right] \equiv T_k^2(a), \tag{4.15}$$

and is discussed in the following section.

The first term describes the growth of fluctuations on large scales, and is usually recast in terms of the (linear) *growth rate* for density fluctuations D_+,

$$\left[\mathcal{D}_+(a_*)^2 \frac{G_p(k\to 0,a)}{G_p(k\to 0,a_*)}\right] \equiv D_+^2(a)$$

(notice that in this form D_+ is also independent of the reference moment a_*). The linear growth rate is normalized so that

$$\lim_{a\to 0} \frac{D_+(a)}{\mathcal{D}_+(a)} = 1,$$

i.e. in the matter+radiation dominated regime $D_+(a) = \mathcal{D}_+(a)$.

The growing mode is the solution of the equation for density fluctuations in the non-relativistic and pressureless medium Peebles (1980),

$$\frac{d^2 D_+}{dt^2} + 2H\frac{dD_+}{dt} = 4\pi G\bar{\rho}_m D_+ \tag{4.16}$$

with initial conditions $D_+(a_*) = \mathcal{D}_+(a_*)$, $\dot{D}_+(a_*) = \dot{\mathcal{D}}_+(a_*)$, where a_* is sufficiently small (any value for $a_* < 0.03$, including arbitrarily small, is enough to obtain better than 10^{-4} precision). For a general case cosmology, this equation needs to be solved numerically.

Thus, the power spectrum for a particular matter component j is

$$P_j(k,a) = P_0(k)\left[D_+(a)T_k^{(j)}(a)\right]^2. \tag{4.17}$$

In a most general case, the primordial power spectrum $P_0(k)$ should be specified in a model for generating primordial fluctuations. For a most commonly used case of inflationary models, the primordial power spectrum is well approximated as a power-law,

$$P_0(k) \propto k^{n_s},$$

where n_s is called a (scalar) spectral index; the case of $n_s = 1$ is usually referred to as (scale-free) Harrison-Zel'dovich spectrum Harrison (1970); Zeldovich (1972). Occasionally, a "running of the spectral index" is also included, i.e. the spectral index n_s is considered a linear function of $\ln(k/k_0)$,

$$n_s(k) = n_s(k_0) + \alpha \ln(k/k_0),$$

where k_0 is a "pivot" point and

$$\alpha \equiv \frac{1}{2}\left.\frac{d^2 \ln P_0(k)}{d\ln k^2}\right|_{k_0}.$$

There exist a number of accurate approximations for $D_+(a)$ for various cosmologies. For example, Carroll, Press & Turner Carroll et al. (1992) give the following approximate formula for the matter + curvature + cosmological constant model:

$$D_+(a) = \frac{5a\omega_m/2}{\omega_m^{4/7} - \omega_\Lambda + (1+\omega_m/2)\,(1+\omega_\Lambda/70)},$$

where $\omega_m \equiv \Omega_m H_0^2/(a^3H^2)$ and $\omega_\Lambda \equiv \Omega_\Lambda H_0^2/H^2$. This formula is not applicable to the dark energy cosmologies, for which the growth factor should be calculated via numerical integration of equation (4.16).

For a restricted cosmological model that only includes matter, curvature, and a cosmological constant (i.e. ignoring radiation and without non-trivial dark energy), the growth rate can be expressed as a quadrature Heath (1977),

$$D_+(a) = \frac{5}{2}H_0^2\Omega_m H(a)\int_0^a \frac{da'}{\left(a'H(a')\right)^3},$$

where $H(a)$ is the Hubble parameter as a function of scale factor a. Nowadays such a cosmology is of no interest, so that solution has only a historic interest.

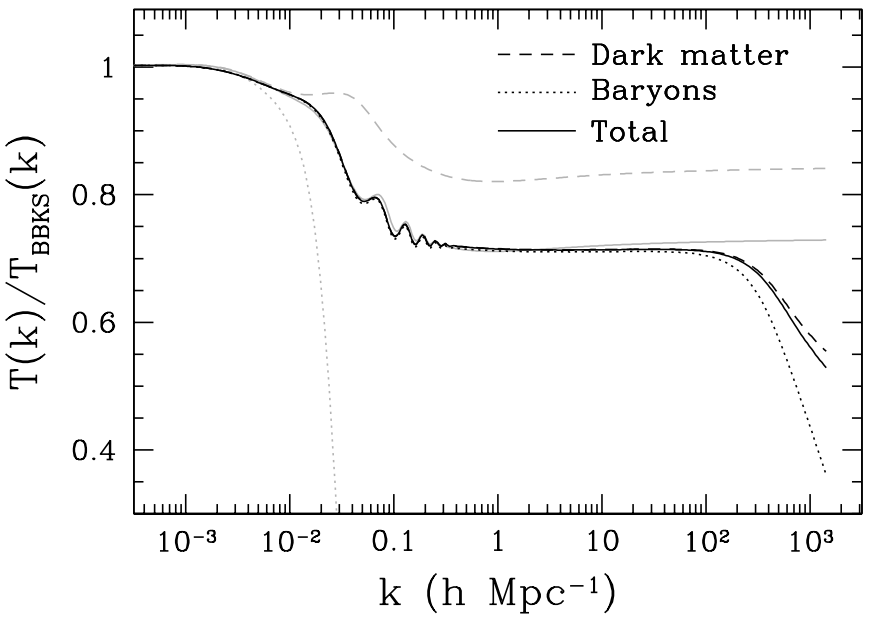


Figure 4.10: An example of transfer functions for the total matter (solid lines), dark matter (dashed lines), and baryons (dotted lines) for a "round number" ΛCDM cosmological model (4.18). The transfer functions are shown as ratios to the smooth BBKS fitting formula with the same cosmological parameters for the exact calculation using a full Boltzmann code (black lines) and for Eisenstein & Hu fits (gray lines).

4.21.2 Transfer Functions

Transfer functions $T_k^{(j)}$ for various matter components have a universal property that

$$T_k^{(j)} \to 1, \quad \text{for } k \to 0.$$

This is simply because no physical process can change the primordial power spectrum on scales outside the cosmic horizon due to causality.

Computing $T_k^{(j)}(a)$ for dark matter, baryons, and massive neutrinos requires running a cosmological Boltzmann code - the same code used for computing the CMB spectrum. While it is not our purpose here to discuss cosmological Boltzmann codes, or give an overview of them, any modern Boltzmann code is sufficient for computing transfer functions to a precision of $< 10^{-3}$ Seljak et al. (2003) within a reasonable amount of time (a few hours at most). For example, both CMBFAST[17]Seljak & Zaldarriaga (1996) and CAMB[18]Lewis et al. (2000) are good codes to use when you need transfer functions with high precision in a large range of k, or for an exotic model that lacks good analytical fitting formulae for $T_k^{(j)}(a)$.

In the standard paradigm of Cold Dark Matter (CDM; with any spatial curvature and any dark energy component) neutrino masses can often be neglected, and baryons are assumed to trace the dark matter on sufficiently large scales (above a few hundred kpc)[19]. In that case only the dark matter transfer function is needed. Before the advent of fast cosmological Boltzmann codes, analytic approximation for the transfer function were in wide use. Of these, the most commonly used are the Bond & Efstathiou approximation Bond & Efstathiou (1984), the Bardeend et al. (or "BBKS") approximation Bardeen et al. (1986) and Eisenstein & Hu approximation Eisenstein & Hu (1998, 1999). The errors of these approximation vary depending on cosmology. The Eisenstein & Hu approximation being the most accurate with errors smaller than a couple per cent at scales $k \lesssim 100h\,\mathrm{Mpc}$ [20] for standard cosmologies without neutrinos, but can be larger for cosmologies with even a modest contribution of neutrinos Kiakotou et al. (2008). As an example,

[17]Current location: `http://cfa-www.harvard.edu/~mzaldarr/CMBFAST/cmbfast.html`
[18]Current location: `http://camb.info`
[19]We discuss the relationship between the baryonic and CDM transfer functions in §4.22
[20]At smaller scales the approximation does not take into account the filtering of the power in the baryon distribution at the Jeans scale.

we show in Fig. 4.10 the transfer functions for the dark matter and baryons for a "round number" ΛCDM cosmological model,

$$\Omega_m = 0.3, \qquad \Omega_\Lambda = 0.7, \qquad \Omega_b = 0.04, \qquad h = 0.7, \qquad n_S = 1. \tag{4.18}$$

It is typical for the BBKS formula to be off by 30%, so, despite its importance for early simulations, it should not be used any more. The Eisenstein & Hu fits are accurate to a few percent for the full matter transfer functions (on sufficiently large scales, $k < 100h\,\mathrm{Mpc}^{-1}$), but should not be used for computing separate transfer functions for the cold dark matter and baryons.

In cases when a per cent precision is required an accurate Boltzmann code should be used. Given that fast and accurate public codes are widely available there is little excuse for resorting to approximations when simulating a specific cosmology.

4.21.3 Normalization

The amplitude of the primordial power spectrum is one of the input cosmological parameters of the simulations. It cannot be predicted from the first principles currently, since it depends on the unknown details of an inflationary model producing fluctuations[21]. One of the widely used, historical[22] ways of normalizing the matter power spectrum was by setting the *rms* density fluctuations in spheres of $R_8 \equiv 8h^{-1}\,\mathrm{Mpc}$ radius, σ_8, which, by Percival theorem, can be related to an integral over the matter power spectrum,

$$\sigma_8^2 = \frac{1}{2\pi^2}\int_0^\infty P(k,a) W_{\rm TH}^2(kR_8) k^2 dk, \tag{4.19}$$

where $W_{\rm TH}(x) \equiv 3(\sin x - x\cos x)/x^3$ is a Fourier transform of a uniform sphere in 3D.

The, σ_8, normalization at the scale of $8h^{-1}\,\mathrm{Mpc}$ can be constrained directly from the data, particularly measurements sensitive to the growth of structure, such as cluster abundance or matter power spectrum probed via weak lensing. Some of the most accurate measurements of the power spectrum normalization, however, are obtained at the largest scales using precision measurements of the CMB temperature fluctuations. This normalization is defined differently.

In the 5th year WMAP analyses, for example, normalization of the primordial matter density fluctuation spectrum, $\Delta_0(k) \equiv k^3 P_0(k)/(2\pi^2)$ is specified as

$$\Delta_0^2(k) = \Delta_{\mathcal{R}}^2 \left(\frac{2c^2k^2}{5\Omega_m H_0^2}\right)^2 \left(\frac{k}{k_0}\right)^{n_s-1}, \tag{4.20}$$

where the pivot point $k_0 = 0.02\,\mathrm{Mpc}^{-1}$ is chosen to provide normalization that is almost independent of other cosmological parameters, and $\Delta_{\mathcal{R}}$ is the primordial curvature perturbation. The WMAP-5 normalization is explicitly presented in the form of

$$\Delta_{\mathcal{R}}^2 = (2.21 \pm 0.09) \times 10^{-9}$$

for models without tensor modes or running spectral index Komatsu et al. (2008).

It does not matter which observational measurement of normalization is used to motivate a choice of normalization for a simulation. Note, however, that if the CMB normalization is used the corresponding normalization at the scales accessible to simulations (e.g., $8h^{-1}\,\mathrm{Mpc}$) will depend on other cosmological parameters, including the Thompson optical depth until the surface of last scattering, τ. For models, in which neutrino contribution to the matter density can be safely neglected, an accurate (to the 1% level) fitting formula relating CMB normalization to σ_8 as a function of other cosmological parameters is given in Hu & Jain (2004) by their equation A5.[23]

4.22 Initial conditions for the baryonic component

Including the gas (alias "baryonic component") in the simulation introduces two complications: (i) the gas temperature needs to be specified, and (ii) baryons and dark matter in general have different transfer functions. Good news is that there is a whole class of problems (including modeling galaxy clusters) for which these details are unimportant. For other problems (like modeling reionization and formation of first galaxies) these details are, on the other hand, completely crucial.

[21] For example, on the functional form of the inflaton potential.

[22] The scale of $8h^{-1}\,\mathrm{Mpc}$ was chosen because rms fluctuation of galaxy overdensity was about unity at these scales in the early studies of galaxy clustering.

[23] Note that the CMB normalization in that paper, is defined at a different pivot scale of $k_0 = 0.05\ \mathrm{Mpc}^{-1}$.

4.22.1 Thermal history of the cosmic gas

Since we have never heard of anyone starting a cosmological simulation before electron - positron annihilation epoch ($a \sim 10^{-9}$), we assume that only Compton scattering with the CMB affects gas temperature. At sufficiently early time the scattering keeps the gas coupled to radiation, so the gas temperature tracks that of CMB at all densities. The temperature decoupling has been worked out in Peebles (1968). The electron temperature of the cosmic gas is governed by the following equation:

$$\frac{dT_e}{dt} = -2HT_e + \frac{8\sigma_T a_R T^4}{3m_e c}\frac{n_e}{n_{\rm tot}}\left(T - T_e\right), \tag{4.21}$$

where $T = 2.725\,{\rm K}/a$ is the CMB temperature, n_e is the number density of free electrons, $n_{\rm tot}$ is the total number density of particles in the plasma, and a combination of fundamental constants $8\sigma_T a_R/(3m_e c) = 4.91395 \times 10^{-22}\,{\rm s^{-1}\,K^{-4}}$.

At later times, when the gas expand adiabatically, the gas temperature evolves with time as

$$T(a) = T_0\frac{a_{\rm td}}{a^2}, \tag{4.22}$$

where $T_0 = 2.725\,{\rm K}$ is the present day CMB temperature, and

$$a_{\rm td} = \frac{1}{137}\left(\frac{\Omega_b h^2(1+\delta)}{0.022}\right)^{-2/5}$$

(the denser gas decouples earlier).

For very high resolution simulations that need to start at really early times ($z > 30$), equation (4.22) is not accurate enough. The following fit to the numerical calculation of the temperature decoupling is accurate to 2% at all times:

$$T(a) = \frac{T_0}{a}\frac{a_{\rm td}}{\left(a^\beta + a_{\rm td}^\beta\right)^{1/\beta}}$$

with $\beta = 1.73$. If higher precision is desired, the equation (4.21) needs to be solved numerically, including the residual time dependence of n_e - even at $z \sim 100$ the electron fraction continues to evolve at a percent level.

4.22.2 Baryonic transfer function

For high accuracy initial consitions, there is no substitute for using the full Boltzmann code for computing the baryonic tranfer function and the power spectrum, as described in § 4.21.2. However, if a 10% precision for the baryonic power spectrum (about 1% precision for the total power spectrum) sufficies, a linear theory filtering scale can be used Gnedin & Hui (1998). The filtering scale describes the difference in the evolution of baryons as compared to the dark matter; it appears in the solution to the coupled linear theory equations for the density fluctuations in the dark matter and baryons. As has been shown in Gnedin et al. (2003), for a wide range of thermal histories, the baryonic power spectrum $P_b(k,a)$ can be obtained as a filtered version of the total matter power spectrum,

$$P_b(k,a) \approx P(k,a)e^{-2k^2/k_F^2},$$

where $k_F(a)$ is the wavenumber of the filtering scale. For standard thermal history, well after the temperature decoupling (but, of course, before reionization), the filtering wavenumber Gnedin & Hui (1998)

$$k_F(a) \approx 0.482\sqrt{\Omega_m}\left(\frac{a/a_{\rm td}}{3\ln(a/a_{\rm td}) - 3 + 4\sqrt{a_{\rm td}/a}}\right)^{1/2} h\,{\rm kpc}^{-1}.$$

If the initial conditions need to be set close to or even before the temperature decoupling time, then the filtering scale is not sufficient. At these times the baryonic transfer evolves in a non-trivial way, because baryons are still catching up to the dark matter after recombination. The dark matter transfer function is also evolves sligtly, responding to the evolution in the baryonic component - even on 100 Mpc scales. If precision of better than 10-20% is required in the initial conditions, there is really no alternative to using the full Boltzmann code for computing the dark matter and baryonic transfer functions.

4.23 Measuring the Power Spectrum

One of the important numerical tasks is measuring the density power spectrum from the particle positions $\vec{x}_i$.

The simplest (and fully sufficient) approach is to start with assigning the particle density on the regular grid, just like in the PM method. The grid does *not* have to be the same size as the PM grid of your code, and the method we describe here works even for high resolution N-body codes.

Given the density field on the regular grid of size N (for simplicity, we assume the cubic grid here, it is straightforward to extend the method to any rectangular grid), $\rho(i,j,k)$, we transform it into Fourier space (preferably, using FFT),

$$\hat{\rho}_{lmn} = \sum_{i=1}^{N}\sum_{j=1}^{N}\sum_{k=1}^{N} \rho(i,j,k) e^{\mathrm{i}\frac{2\pi}{N}(il+jm+kn)}.$$

In order to compute the power spectrum, we need to average all individual wavevectors that correspond to a given wavenumber k, or, more precisely, which fall into a given range of wavenumbers. Namely, let's introduce a 1D grid of wavenumbers k_M, $M = 1, ...$, which may be linearly spaces, logarithmically spaced, or have any other sampling scheme. Then for each wavevector on the grid,

$$\vec{k}_{lmn} = \frac{2\pi}{L}(l,m,n)$$

where L is the size of the cubic grid, we can compute the bin M into which this wavevector falls,

$$k_M \le |\vec{k}_{lmn}| = \frac{2\pi}{L}\sqrt{l^2+m^2+n^2} < k_{M+1}.$$

Then, the power spectrum $P(k)$ in the wavenumber bin $k_M \le k < k_{M+1}$ is just

$$P(k)|_{k_M \le k < k_{M+1}} = \langle |\hat{\rho}_{lmn}|^2 \rangle_M$$

where averaging $\langle\rangle_M$ is done over all wavevectors that fall into the wavenumver bin $k_M \le |\vec{k}_{lmn}| < k_{M+1}$.

In this approach the largest value of the wavenumber one can sample is determined by the Nyquist frequency of the adopted grid, $k_{\max} = \pi N/L$. What if this wavenumber is too small for our purposes - for example, we have a high resolution simulations that fairly models a dynamic range of 30,000 - using a $30{,}000^3$ for our density assignment is not very practical. In this case there is a trick that comes to our rescue (Jenkins et al., 1998, see also section 5.1 of Kravtsov & Klypin 1999 for a somewhat more detailed description of the technique). Just choose some reasonable N that you can use on the machine you are working on, say $N = 1024$. With this method we will sample the power spectrum from the fundamental wavenumber of the box, $k_1 = 2\pi/L$, to $k_{512} = \pi N/L = 512 k_1$, which is a factor of 60 short of our goal of the 30,000 dynamic range.

Now do the following: cut your simulation box into 8 octants, and move each along octant into the first one. In other words, replace each particle position $\vec{x}_i \equiv \vec{x}_i^{(1)}$ with

$$\vec{x}_i^{(2)} = L\Psi\left(\frac{\vec{x}_i}{L}\right),$$

where function Ψ is defined as

$$\Psi(t) = \begin{cases} t, & \text{if } 0 \le t < 1/2, \text{ and} \\ t - 1/2, & \text{if } 1/2 \le t < 1. \end{cases}$$

Now assign the density of particles with positions $\vec{x}_i^{(2)}$ on the grid of size N and recompute the power spectrum. As if by miracle, you now computed the power spectrum for the wavenumber range $k_2 \le k \le k_{1024}$. As you repeat this procedure over and over again, in each iteration n wrapping particle positions

$$\vec{x}_i^{(n+1)} = L\Psi\left(\frac{\vec{x}_i^{(n)}}{L}\right),$$

you successively compute the power spectrum in the range of wavenumbers from k_{2^n} to $k_{2^{(n+9)}}$, and in 6 iterations you reach the desired wavenumber $k_{32,768}$. All you have to do now is to collect 6 different power spectra, each over a different wavenumber range (first from k_1 to k_{512}, second from k_2 to k_{1024}, etc) into a single measurement of the power spectrum that spans a factor of 30,000 in dynamic range.

Bibliography

Aarseth, S. J. 1963, MNRAS, 126, 223

—. 2003, Gravitational N-Body Simulations

Aarseth, S. J., Turner, E. L., & Gott, III, J. R. 1979, ApJ, 228, 664

Abel, T., Hahn, O., & Kaehler, R. 2012, MNRAS, 427, 61

Bardeen, J. M., Bond, J. R., Kaiser, N., & Szalay, A. S. 1986, ApJ, 304, 15

Barnes, J., & Hut, P. 1986, Nature, 324, 446

Baugh, C. M., Gaztanaga, E., & Efstathiou, G. 1995, MNRAS, 274, 1049

Bertschinger, E. 1987, ApJ, 323, L103

—. 1998, ARA&A, 36, 599

—. 2001, ApJS, 137, 1

—. 2011, GRAFIC-2: Multiscale Gaussian Random Fields for Cosmological Simulations, astrophysics Source Code Library (arxiv/1106.008)

Binney, J., & Tremaine, S. 2008, Galactic Dynamics: Second Edition (Princeton University Press)

Bode, P., Ostriker, J. P., & Xu, G. 2000, ApJS, 128, 561

Bond, J. R., & Efstathiou, G. 1984, ApJ, 285, L45

Bryan, G. L., & Norman, M. L. 1997, ArXiv Astrophysics e-prints

Byers, J., & Grewal, M. 1970, Phys. Fluids, 13, 1819

Carroll, S. M., Press, W. H., & Turner, E. L. 1992, ARA&A, 30, 499

Colin, P., Carlberg, R. G., & Couchman, H. M. P. 1997, ApJ, 490, 1

Couchman, H. M. P. 1991, ApJ, 368, L23

Crocce, M., Pueblas, S., & Scoccimarro, R. 2006, MNRAS, 373, 369

Doroshkevich, A. G., Kotok, E. V., Poliudov, A. N., Shandarin, S. F., Sigov, I. S., & Novikov, I. D. 1980, MNRAS, 192, 321

Efstathiou, G. 1979, MNRAS, 187, 117

Efstathiou, G., Davis, M., White, S. D. M., & Frenk, C. S. 1985, ApJS, 57, 241

Efstathiou, G., & Eastwood, J. W. 1981, MNRAS, 194, 503

Eisenstein, D. J., & Hu, W. 1998, ApJ, 496, 605

—. 1999, ApJ, 511, 5

Fall, S. M. 1978, MNRAS, 185, 165

Gnedin, N. Y. 1995, ApJS, 97, 231

Gnedin, N. Y., & Bertschinger, E. 1996, ApJ, 470, 115

Gnedin, N. Y., & Hui, L. 1998, MNRAS, 296, 44

Gnedin, N. Y., et al. 2003, ApJ, 583, 525

Götz, M., & Sommer-Larsen, J. 2003, Ap&SS, 284, 341

Hahn, O., Abel, T., & Kaehler, R. 2013, MNRAS

Hansen, S. H., Agertz, O., Joyce, M., Stadel, J., Moore, B., & Potter, D. 2007, ApJ, 656, 631

Harrison, E. R. 1970, Phys. Rev. D, 1, 2726

Heath, D. J. 1977, MNRAS, 179, 351

Heggie, D., & Hut, P. 2003, The Gravitational Million-Body Problem: A Multidisciplinary Approach to Star Cluster Dynamics

Hockney, R. W., & Eastwood, J. W. 1981, Computer Simulation Using Particles

Hoffman, Y., & Ribak, E. 1991, ApJ, 380, L5

Holmberg, E. 1941, ApJ, 94, 385

Hu, W., & Jain, B. 2004, Phys. Rev. D, 70, 043009

Jenkins, A., et al. 1998, ApJ, 499, 20

Joyce, M., & Marcos, B. 2007, Phys. Rev. D, 75, 063516

Kiakotou, A., Elgarøy, Ø., & Lahav, O. 2008, Phys. Rev. D, 77, 063005

Klypin, A., Kravtsov, A. V., Bullock, J. S., & Primack, J. R. 2001, ApJ, 554, 903

Klypin, A. A., & Shandarin, S. F. 1983, MNRAS, 204, 891

Kolb, E. W., Chung, D. J. H., & Riotto, A. 1999, in American Institute of Physics Conference Series, Vol. 484, Trends in Theoretical Physics II, ed. H. Falomir, R. E. Gamboa Saravi, & F. A. Schaposnik, 91–105

Komatsu, E., et al. 2008, ArXiv e-prints, 803

Kravtsov, A. V., & Klypin, A. A. 1999, ApJ, 520, 437

Kravtsov, A. V., Klypin, A. A., & Khokhlov, A. M. 1997, ApJS, 111, 73

Lewis, A., Challinor, A., & Lasenby, A. 2000, ApJ, 538, 473

Loeb, A., & Weiner, N. 2011, Physical Review Letters, 106, 171302

Makino, J. 2008, in American Institute of Physics Conference Series, Vol. 1011, New Facet of Three Nucleon Force - 50 Years of Fujita Miyazawa, ed. H. Sakai, K. Sekiguchi, & B. F. Gibson, 199–208

Miller, R. H. 1983, ApJ, 270, 390

—. 1984, A&A, 138, 121

Neyrinck, M. C., Hamilton, A. J. S., & Gnedin, N. Y. 2004, MNRAS, 348, 1

Peebles, P. J. E. 1968, ApJ, 153, 1

—. 1970, AJ, 75, 13

—. 1980, The large-scale structure of the universe (Research supported by the National Science Foundation. Princeton, N.J., Princeton University Press, 1980. 435 p.)

Pen, U.-L. 1997, ApJ, 490, L127

—. 1998, ApJS, 115, 19

Roszkowski, L. 1999, in American Institute of Physics Conference Series, Vol. 478, COSMO-98, ed. D. O. Caldwell, 316–324

Salmon, J. 1996, ApJ, 460, 59

Seljak, U., Sugiyama, N., White, M., & Zaldarriaga, M. 2003, Phys. Rev. D, 68, 083507

Seljak, U., & Zaldarriaga, M. 1996, ApJ, 469, 437

Sellwood, J. A. 1983, Journal of Computational Physics, 50, 337

Sirko, E. 2005, ApJ, 634, 728

Smith, R. E., et al. 2003, MNRAS, 341, 1311

Spergel, D. N., & Steinhardt, P. J. 2000, Physical Review Letters, 84, 3760

Suisalu, I., & Saar, E. 1995, MNRAS, 274, 287

Teyssier, R. 2002, A&A, 385, 337

Villumsen, J. V. 1989, ApJS, 71, 407

von Hoerner, S. 1960, Zeitschrift für Astrophysik, 50, 184

Wang, J., & White, S. D. M. 2007, MNRAS, 380, 93

White, S. D. M. 1996, in Cosmology and Large Scale Structure, ed. R. Schaeffer, J. Silk, M. Spiro, & J. Zinn-Justin, 349

Xu, G. 1995, ApJS, 98, 355

Zel'dovich, Y. B. 1965, Adv. Astron. & Astrophys., 3, 241

Zeldovich, Y. B. 1972, MNRAS, 160, 1P